

# 1.6W 字！梳理 50 道经典计算机网络面试题（收藏版）

## 前言

金九银十马上就要到了，整理了 50 道计算机网络面试题，每一道都非常的经典，大厂也非常喜欢问。希望大家看完后，都能找到理想的 offer 哈~

- 公众号：捡田螺的小男孩

## 1. HTTP 常用的请求方式，区别和用途？

- **GET**: 发送请求，获取服务器数据
- **POST**: 向 URL 指定的资源提交数据
- **PUT**: 向服务器提交数据，以修改数据
- **HEAD**: 请求页面的首部，获取资源的元信息
- **DELETE**: 删除服务器上的某些资源。
- **CONNECT**: 建立连接隧道，用于代理服务器；
- **OPTIONS**: 列出可对资源实行的请求方法，常用于跨域
- **TRACE**: 追踪请求-响应的传输路径

## 2. HTTP 常用的状态码及含义？

- **1xx**: 接受的请求正在处理 （信息性状态码）
- **2xx**: 表示请求正常处理完毕 （成功状态码）
- **3xx**: 表示重定向状态，需要重新请求 （重定向状态码）
- **4xx**: 服务器无法处理请求 （客户端错误状态码）
- **5xx**: 服务器处理请求出错 （服务端错误状态码）

常用状态码如下：

- **101** 切换请求协议，从 HTTP 切换到 WebSocket
- **200** 请求成功，表示正常返回信息。
- **301** 永久重定向，会缓存
- **302** 临时重定向，不会缓存
- **400** 请求错误
- **403** 服务器禁止访问
- **404** 找不到与 URI 相匹配的资源。
- **500** 常见的服务器端错误

### 3. 从浏览器地址栏输入 url 到显示主页的过程

1. DNS 解析，查找真正的 ip 地址
2. 与服务器建立 TCP 连接
3. 发送 HTTP 请求
4. 服务器处理请求并返回 HTTP 报文
5. 浏览器解析渲染页面
6. 连接结束

| 过程                                             | 使用的协议                                                                                                                                                                     |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.浏览器查找域名的IP地址<br>(DNS查找过程: 浏览器缓存、路由器缓存、DNS缓存) | DNS: 获取域名对应的IP                                                                                                                                                            |
| 2.浏览器向web服务器发送一个HTTP请求<br>(cookies会随着请求发送给服务器) | TCP:与服务器建立TCP连接<br>IP: 建立TCP协议时, 需要发送数据, 发送的数据在网络层使用IP协议<br>OPSF:IP数据包在路由器之间, 路由选择使用OPSF协议<br>ARP:路由器在与服务器通信时, 需要将IP地址转换为MAC地址, 需要适应ARP协议<br>HTTP:在TCP连接完成后, 使用HTTP协议访问网页 |
| 3.服务器处理请求<br>(处理请求 & 它的参数、cookies、生成一个HTML响应)  |                                                                                                                                                                           |
| 4.服务器发回一个HTML响应                                |                                                                                                                                                                           |
| 5.浏览器开始显示HTML                                  |                                                                                                                                                                           |

@掘金技术社区

### 4. 如何理解 HTTP 协议是无状态的

每次 HTTP 请求都是独立的，无相关的，默认不需要保存上下文信息的。我们来看个便于理解的例子：

有状态：

- A: 今天吃啥子？
- B: 罗非鱼！
- A: 味道怎么样呀？
- B: 还不错，好香。

无状态：

- A: 今天吃啥子？
- B: 罗非鱼！
- A: 味道怎么样呀？
- B: ？啊？啥？什么鬼？什么味道怎么样？

加下 cookie 这玩意：

- A: 今天吃啥子?
- B: 罗非鱼
- A: 你今天吃的罗非鱼味道怎么样呀?
- B: 还不错, 好香。

## 5. HTTP 1.0, 1.1, 2.0 的版本区别

### HTTP 1.0

- HTTP 1.0 规定浏览器与服务器只保持**短暂的连接**, 浏览器的每次请求都需要与服务器建立一个 TCP 连接, 服务器完成请求处理后立即断开 TCP 连接。它也可以强制开启长链接, 例如设置 `Connection: keep-alive` 这个字段

### HTTP 1.1

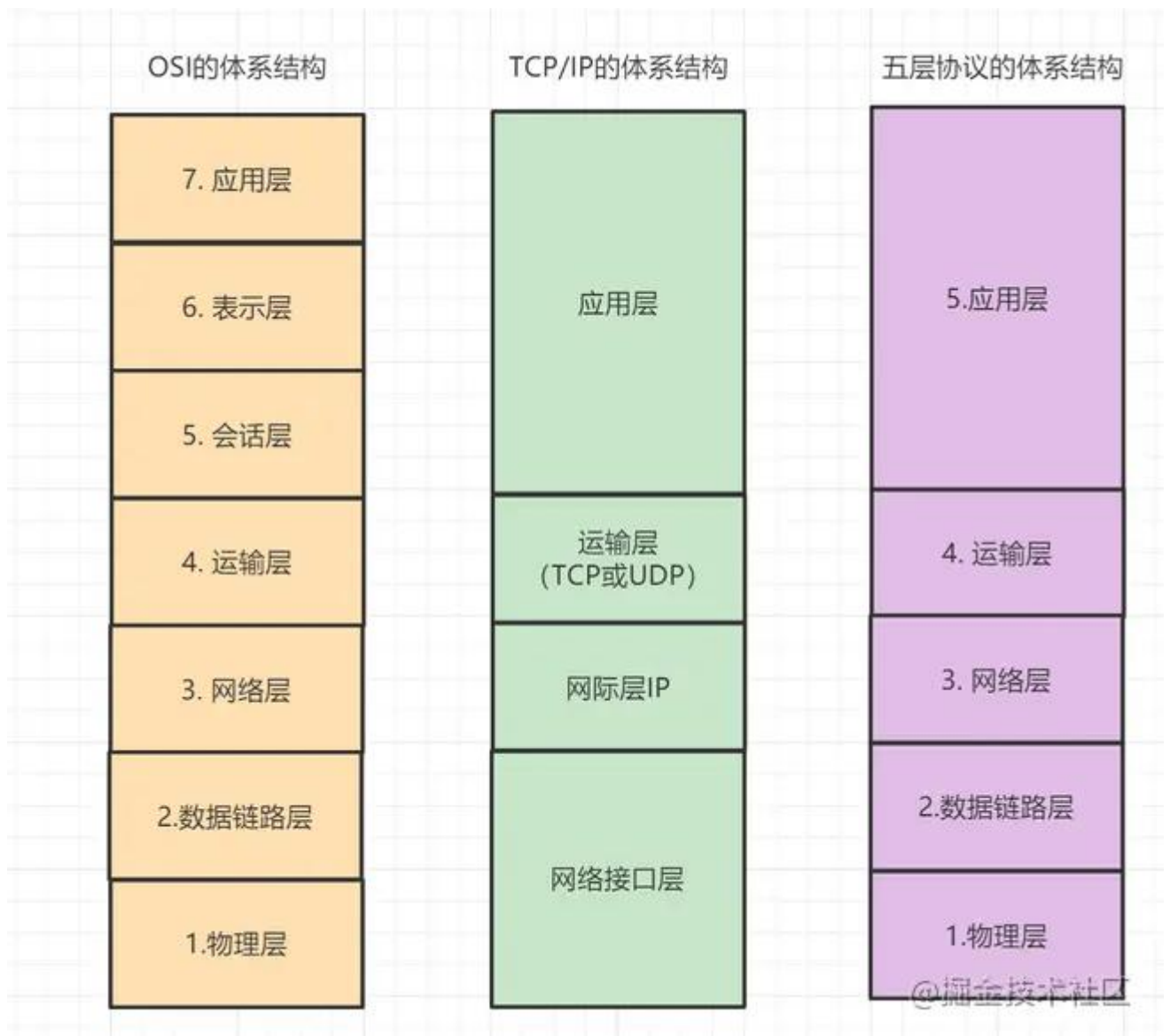
- 引入了长连接, 即 **TCP 连接默认不关闭, 可以被多个请求复用**。
- 引入了管道机制 (**pipelining**), 即在同一个 TCP 连接里面, 客户端可以同时发送多个请求。
- 缓存处理, 引入了更多的缓存控制策略, 如 `Cache-Control`、`Etag/If-None-Match` 等。
- 错误状态管理, 新增了 24 个错误状态响应码, 如 409 表示请求的资源与资源的当前状态发生冲突。

### HTTP 2

- 采用了**多路复用**, 即在一个连接里, 客户端和浏览器都可以同时发送多个请求或回应, 而且不用按照顺序一一对应。
- 服务端推送, HTTP 2 允许服务器未经请求, 主动向客户端发送资源

## 6. 说下计算机网络体系结构

计算机网路体系结构主要有 ISO 七层模型、TCP/IP 四层模型、五层体系结构



## ISO 七层模型

ISO 七层模型是国际标准化组织（ISO）制定的一个用于计算机或通信系统间互联的标准体系。

- 应用层：网络服务与最终用户的一个接口，协议有：**HTTP FTP TFTP SMTP SNMP DNS TELNET HTTPS POP3 DHCP**
- 表示层：数据的表示、安全、压缩。
- 会话层：建立、管理、终止会话。对应主机进程，指本地主机与远程主机正在进行的会话
- 传输层：定义传输数据的协议端口号，以及流控和差错校验。协议有：**TCP UDP，数据包一旦离开网卡即进入网络传输层**
- 网络层：进行逻辑地址寻址，实现不同网络之间的路径选择。协议有：**ICMP IGMP IP (IPv4 IPv6)**
- 数据链路层：建立逻辑连接、进行硬件地址寻址、差错校验等功能。
- 物理层：建立、维护、断开物理连接。

## TCP/IP 四层模型

- 应用层：对应于 OSI 参考模型的（应用层、表示层、会话层），为用户提供所需要的各种服务，例如：FTP、Telnet、DNS、SMTP 等
- 传输层：对应 OSI 的传输层，为应用层实体提供端到端的通信功能，保证了数据包的顺序传送及数据的完整性。定义了 TCP 和 UDP 两层协议。
- 网际层：对应于 OSI 参考模型的网络层，主要解决主机到主机的通信问题。三个主要协议：网际协议（IP）、互联网组管理协议（IGMP）和互联网控制报文协议（ICMP）
- 网络接口层：与 OSI 参考模型的数据链路层、物理层对应。它负责监视数据在主机和网络之间的交换。

## 五层体系结构

- 应用层：通过应用进程间的交互来完成特定网络应用。对应于 OSI 参考模型的（应用层、表示层、会话层），应用层协议很多，如**域名系统 DNS**，HTTP 协议，支持电子邮件的**SMTP 协议**等等。我们把应用层交互的数据单元称为报文。
- 传输层：负责向两台主机进程之间的通信提供通用的数据传输服务。对应 OSI 参考模型的传输层，协议有传输控制协议 TCP 和 用户数据协议 UDP。
- 网络层：对应 OSI 参考模型的的网络层
- 数据链路层：对应 OSI 参考模型的的数据链路层
- 物理层：对应 OSI 参考模型的物理层。在物理层上所传送的数据单位是比特。物理层(physical layer)的作用是实现相邻计算机节点之间比特流的透明传送，尽可能屏蔽掉具体传输介质和物理设备的差异。

## 7. POST 和 GET 有哪些区别？

- **请求参数**：GET 把参数包含在 URL 中，用&连接起来；POST 通过 request body 传递参数。
- **请求缓存**：GET 请求会被主动 Cache，而 POST 请求不会，除非手动设置。
- **收藏为书签**：GET 请求支持收藏为书签，POST 请求不支持。
- **安全性**：POST 比 GET 安全，GET 请求在浏览器回退时是无害的，而 POST 会再次请求。
- **历史记录**：GET 请求参数会被完整保留在浏览历史记录里，而 POST 中的参数不会被保留。
- **编码方式**：GET 请求只能进行 url 编码，而 POST 支持多种编码方式。
- **参数数据类型**：GET 只接受 ASCII 字符，而 POST 没有限制数据类型。
- **数据包**：GET 产生一个 TCP 数据包；POST 可能产生两个 TCP 数据包。

## 8. 在交互过程中如果数据传送完了，还不想断开连接怎么办，怎么维持？

在 HTTP 中响应体的 Connection 字段指定为 keep-alive

## 9. HTTP 如何实现长连接？在什么时候会超时？

### HTTP 如何实现长连接？

- HTTP 分为长连接和短连接，其实本质上说的是 **TCP 的长短连接**。TCP 连接是一个双向的通道，它是可以保持一段时间不关闭的，因此 TCP 连接才有真正的长连接和短连接这一个说法。
- 长连接是指的是 TCP 连接，而不是 HTTP 连接。
- TCP 长连接可以复用一個 TCP 连接来发起多次 HTTP 请求，这样可以减少资源消耗，比如一次请求 HTML，短连接可能还需要请求后续的 JS/CSS/图片等

要实现 HTTP 长连接，在响应头设置 **Connection 为 keep-alive**，HTTP1.1 默认是长连接，而 HTTP 1.0 协议也支持长连接，但是默认是关闭的。

### 在什么时候会超时呢？

- HTTP 一般会有 httpd 守护进程，里面可以设置 keep-alive timeout，当 tcp 链接闲置超过这个时间就会关闭，也可以在 HTTP 的 header 里面设置超时时间
  - TCP 的 keep-alive 包含三个参数，支持在系统内核的 net.ipv4 里面设置：当 TCP 连接之后，闲置了 tcp\_keepalive\_time，则会发生探测包，如果没有收到对方的 ACK，那么会每隔 tcp\_keepalive\_intvl 再发一次，直到发送了 tcp\_keepalive\_probes，就会丢弃该连接。
1. tcp\_keepalive\_intvl = 15
  2. tcp\_keepalive\_probes = 5
  3. tcp\_keepalive\_time = 1800

## 10. 讲一下 HTTP 与 HTTPS 的区别。

HTTP，超文本传输协议，英文是 **Hyper Text Transfer Protocol**，是一个基于 TCP/IP 通信协议来传递数据的协议。HTTP 存在这几个问题：

- 请求信息明文传输，容易被窃听截取。
- 数据的完整性未校验，容易被篡改
- 没有验证对方身份，存在冒充危险

HTTPS 就是为了解决 HTTP 存在问题的。HTTPS，英文是 HyperText Transfer Protocol over Secure Socket Layer，可以这么理解 Https 是身披 SSL(Secure Socket Layer)的 HTTP，即 **HTTPS 协议 = HTTP+SSL/TLS**。通过 SSL 证书来验证服务器的身份，并为浏览器和服务器之间的传输数据进行加密。

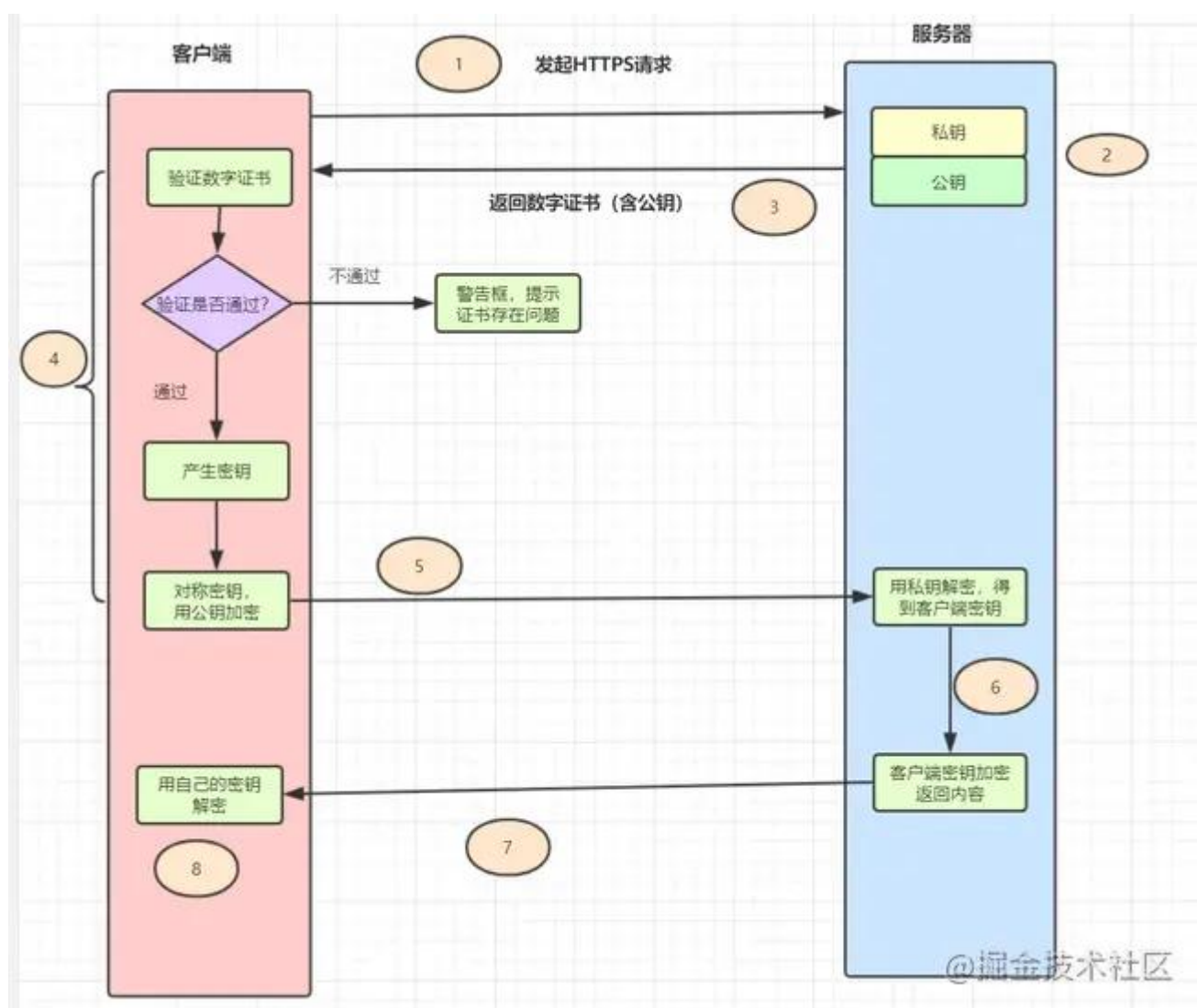
它们主要区别：

- 数据是否加密: Http 是明文传输，HTTPS 是密文

- 默认端口: Http 默认端口是 80, Https 默认端口是 443
- 资源消耗: 和 HTTP 通信相比, Https 通信会消耗更多的 CPU 和内存资源, 因为需要加解密处理;
- 安全性: http 不安全, https 比较安全。

## 11 . Https 流程是怎样的?

- HTTPS = HTTP + SSL/TLS, 即用 SSL/TLS 对数据进行加密和解密, Http 进行传输。
- SSL, 即 Secure Sockets Layer (安全套接层协议), 是网络通信提供安全及数据完整性的一种安全协议。
- TLS, 即 Transport Layer Security(安全传输层协议), 它是 SSL 3.0 的后续版本。

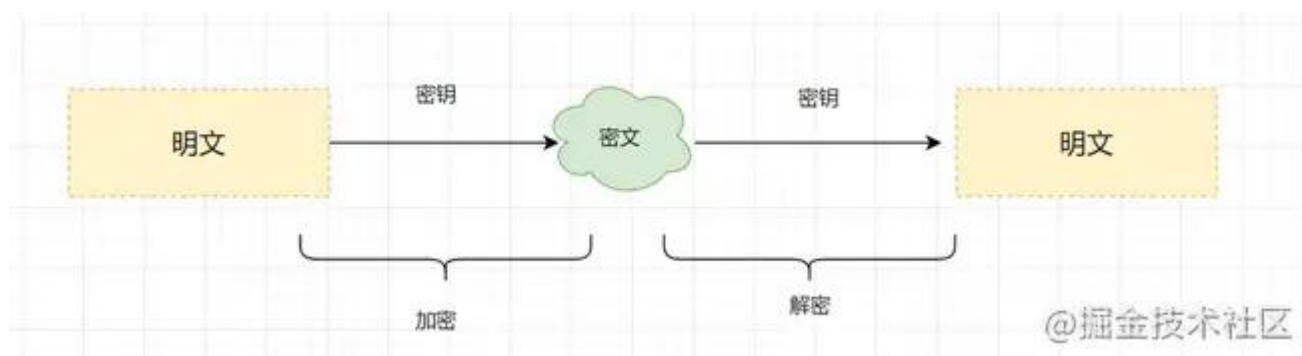


1. 用户在浏览器里输入一个 https 网址, 然后连接到 server 的 443 端口。
2. 服务器必须要有一套数字证书, 可以自己制作, 也可以向组织申请, 区别就是自己颁发的证书需要客户端验证通过。这套证书其实就是一对公钥和私钥。
3. 服务器将自己的数字证书（含有公钥）发送给客户端。

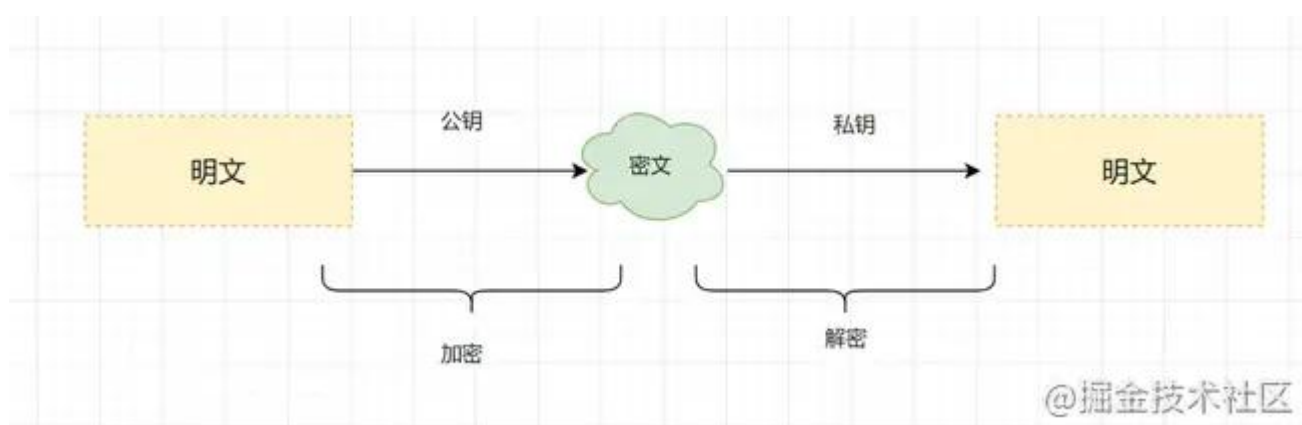
4. 客户端收到服务器端的数字证书之后，会对其进行检查，如果不通过，则弹出警告框。如果证书没问题，则生成一个密钥（对称加密），用证书的公钥对它加密。
5. 客户端会发起 HTTPS 中的第二个 HTTP 请求，将加密之后的客户端密钥发送给服务器。
6. 服务器接收到客户端发来的密文之后，会用自己的私钥对其进行非对称解密，解密之后得到客户端密钥，然后用客户端密钥对返回数据进行对称加密，这样数据就变成了密文。
7. 服务器将加密后的密文返回给客户端。
8. 客户端收到服务器发返回的密文，用自己的密钥（客户端密钥）对其进行对称解密，得到服务器返回的数据。

## 12. 对称加密与非对称加密有什么区别

对称加密：加密和解密使用相同密钥的加密算法。



非对称加密：非对称加密算法需要两个密钥（公开密钥和私有密钥）。公钥与私钥是成对存在的，如果用公钥对数据进行加密，只有对应的私钥才能解密。

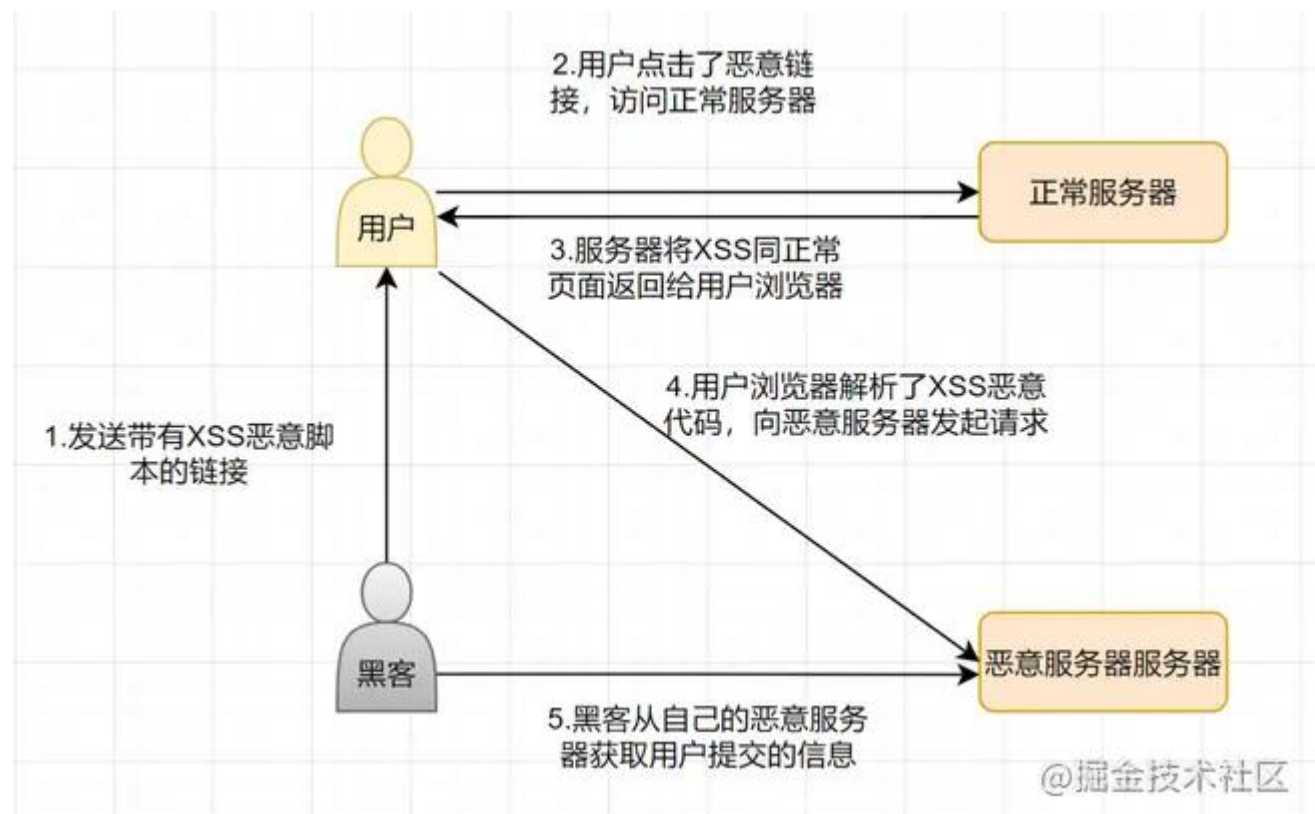


## 13. 什么是 XSS 攻击，如何避免？

XSS 攻击, 全称**跨站脚本攻击 (Cross-Site Scripting)**, 这会与层叠样式表(Cascading Style Sheets, CSS)的缩写混淆, 因此有人将跨站脚本攻击缩写为 XSS。它指的是恶意攻击者往 Web 页面里插入恶意 html 代码, 当用户浏览该页之时, 嵌入其中 Web 里面的 html 代码会被执行, 从而达到恶意攻击用户的特殊目的。XSS 攻击一般分三种类型: **存储型**、**反射型**、**DOM 型 XSS**

### XSS 是如何攻击的?

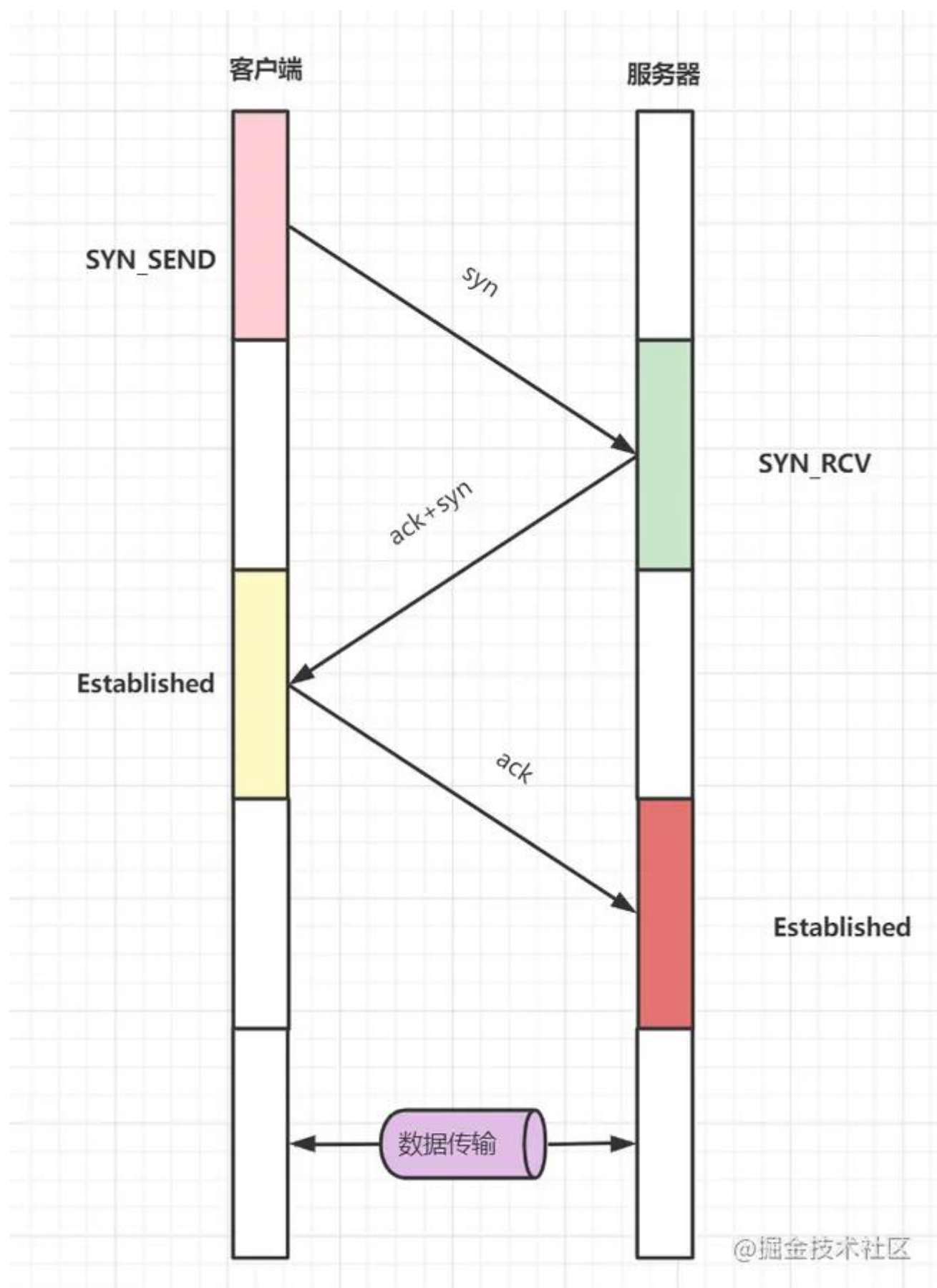
拿反射型举个例子吧, 流程图如下:



### 如何解决 XSS 攻击问题

- 不相信用户的输入, 对输入进行过滤, 过滤标签等, 只允许合法值。
- HTML 转义
- 对于链接跳转, 如 `<a href="xxx">` 等, 要校验内容, 禁止以 `script` 开头的非法链接。
- 限制输入长度等等

### 14. 请详细介绍一下 TCP 的三次握手机制



开始客户端和服务端都处于 CLOSED 状态，然后服务端开始监听某个端口，进入 LISTEN 状态

- 第一次握手(SYN=1, seq=x)，发送完毕后，客户端进入 SYN\_SEND 状态
- 第二次握手(SYN=1, ACK=1, seq=y, ACKnum=x+1)，发送完毕后，服务器端进入 SYN\_RCV 状态。
- 第三次握手(ACK=1, ACKnum=y+1)，发送完毕后，客户端进入 ESTABLISHED 状态，当服务器端接收到这个包时

## 15. TCP 握手为什么是三次，不能是两次？不能是四次？

TCP 握手为什么是三次呢？为了方便理解，我们以谈恋爱为个例子：两个人能走到一起，最重要的事情就是相爱，就是**我爱你，并且我知道，你也爱我**，接下来我们以此来模拟三次握手的过程：

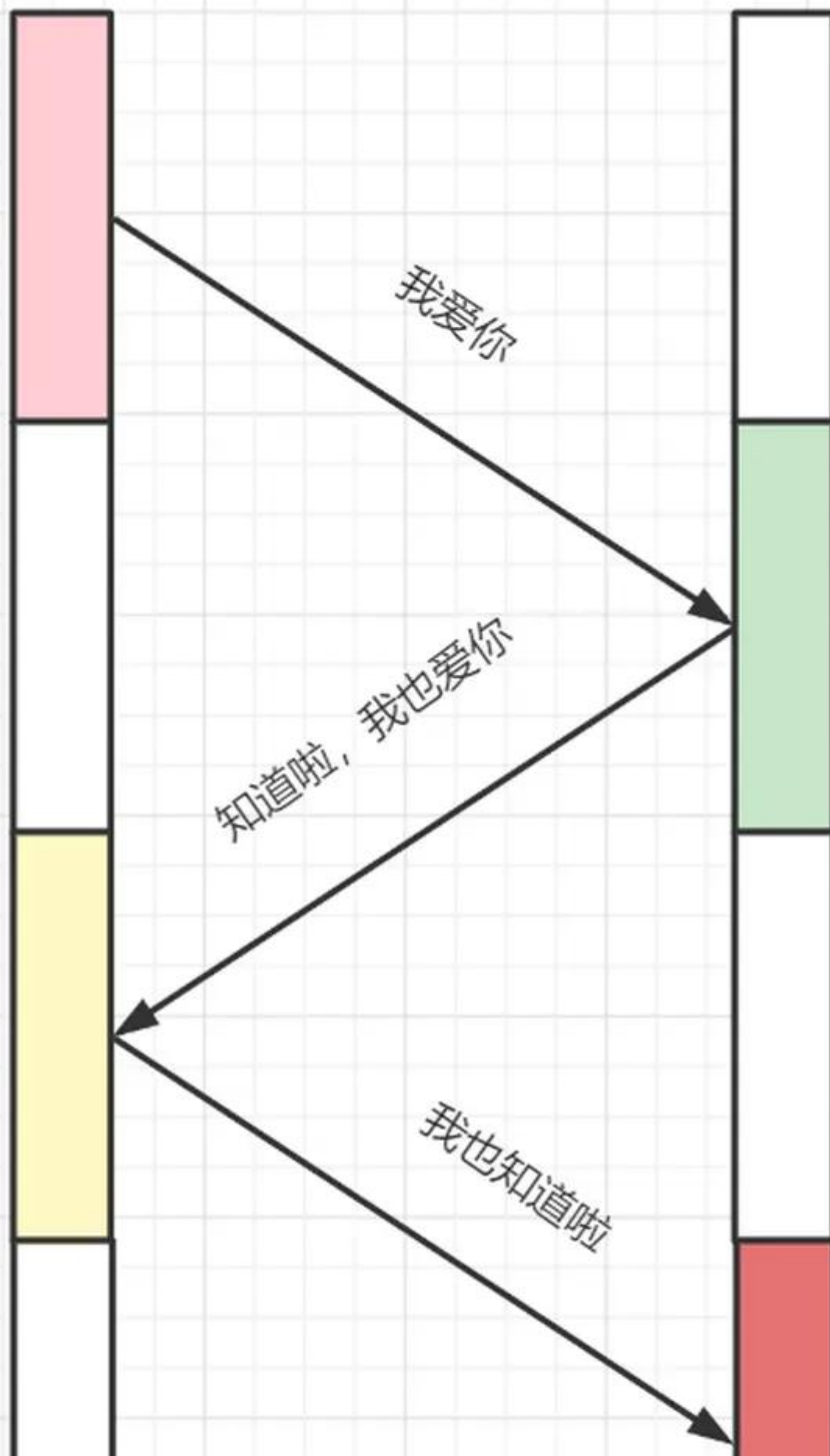
男孩子

女孩子

我爱你

知道啦，我也爱你

我也知道啦



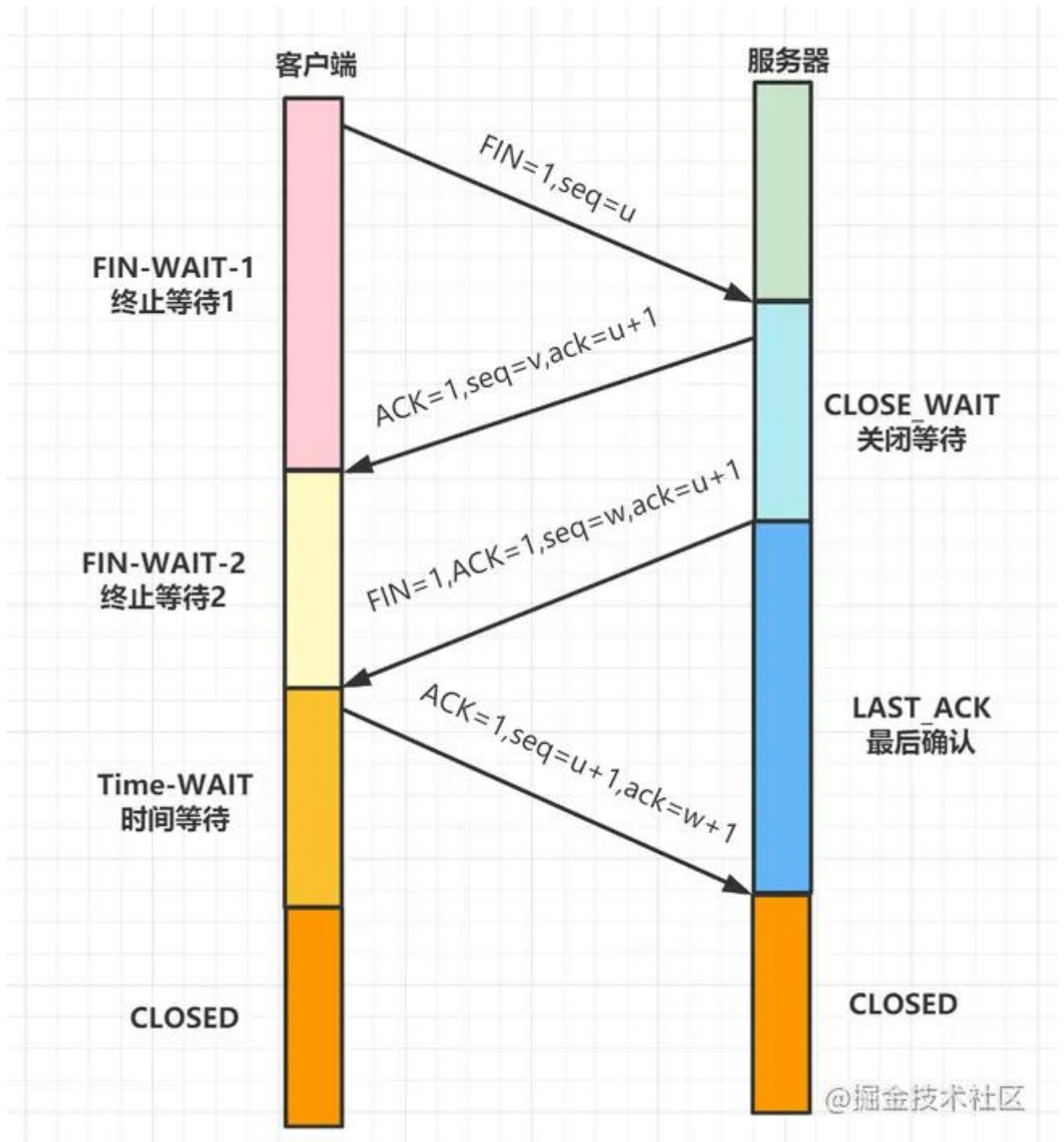
**为什么握手不能是两次呢？**

如果只有两次握手，女孩子可能就不知道，她的那句**我也爱你**，男孩子是否**收到**，恋爱关系就不能愉快展开。

**为什么握手不能是四次呢？**

因为握手不能是四次呢？因为三次已经够了，三次已经能让双方都知道：你爱我，我也爱你。而四次就多余了。

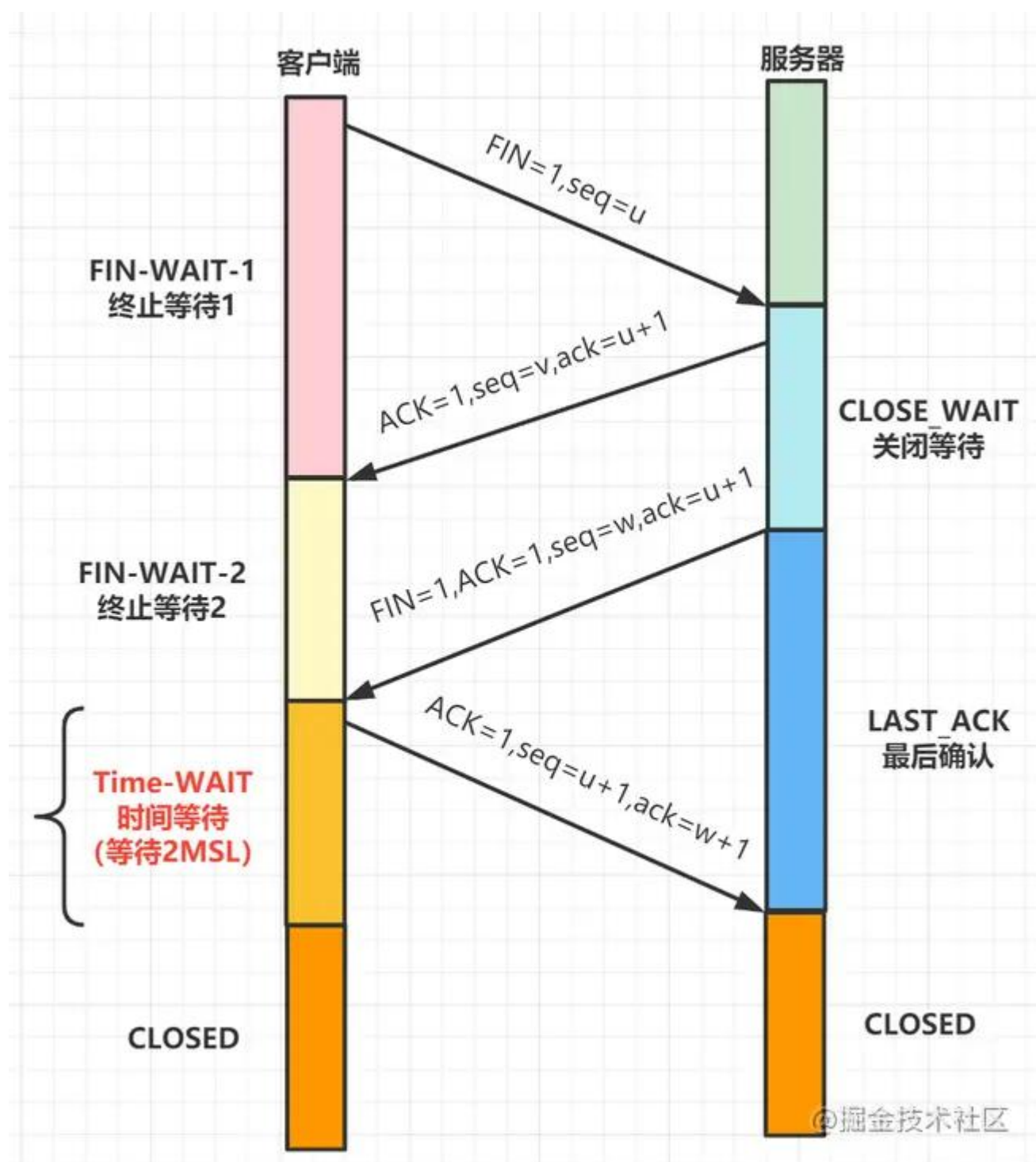
**16. TCP 四次挥手过程？**



1. 第一次挥手(`FIN=1, seq=u`), 发送完毕后, 客户端进入 `FIN_WAIT_1` 状态
2. 第二次挥手(`ACK=1, ack=u+1, seq=v`), 发送完毕后, 服务器端进入 `CLOSE_WAIT` 状态, 客户端接收到这个确认包之后, 进入 `FIN_WAIT_2` 状态
3. 第三次挥手(`FIN=1, ACK=1, seq=w, ack=u+1`), 发送完毕后, 服务器端进入 `LAST_ACK` 状态, 等待来自客户端的最后一个 ACK。
4. 第四次挥手(`ACK=1, seq=u+1, ack=w+1`), 客户端接收到来自服务器端的关闭请求, 发送一个确认包, 并进入 `TIME_WAIT` 状态, 等待了某个固定时间 (两个最大段生命周期, **2MSL, 2 Maximum Segment Lifetime**) 之后, 没有收到服务器端的 `ACK`, 认为

服务器端已经正常关闭连接，于是自己也关闭连接，进入 CLOSED 状态。服务器端接收到这个确认包之后，关闭连接，进入 CLOSED 状态。

## 17. TCP 四次挥手过程中，客户端为什么需要等待 2MSL, 才进入 CLOSED 状态



2MSL, 2 Maximum Segment Lifetime, 即两个最大段生命周期

- 1 个 MSL 保证四次挥手中主动关闭方最后的 ACK 报文能最终到达对端
- 1 个 MSL 保证对端没有收到 ACK 那么进行重传的 FIN 报文能够到达

## 18. 为什么需要四次挥手?

举个例子吧

小明和小红打电话聊天, 通话差不多要结束时, 小红说“我没啥要说的了”, 小明回答“我知道了”。但是小明可能还会有要说的话, 小红不能要求小明跟着自己的节奏结束通话, 于是小明可能又叽叽歪歪说了一通, 最后小明说“我说完了”, 小红回答“知道了”, 这样通话才算结束。

小红

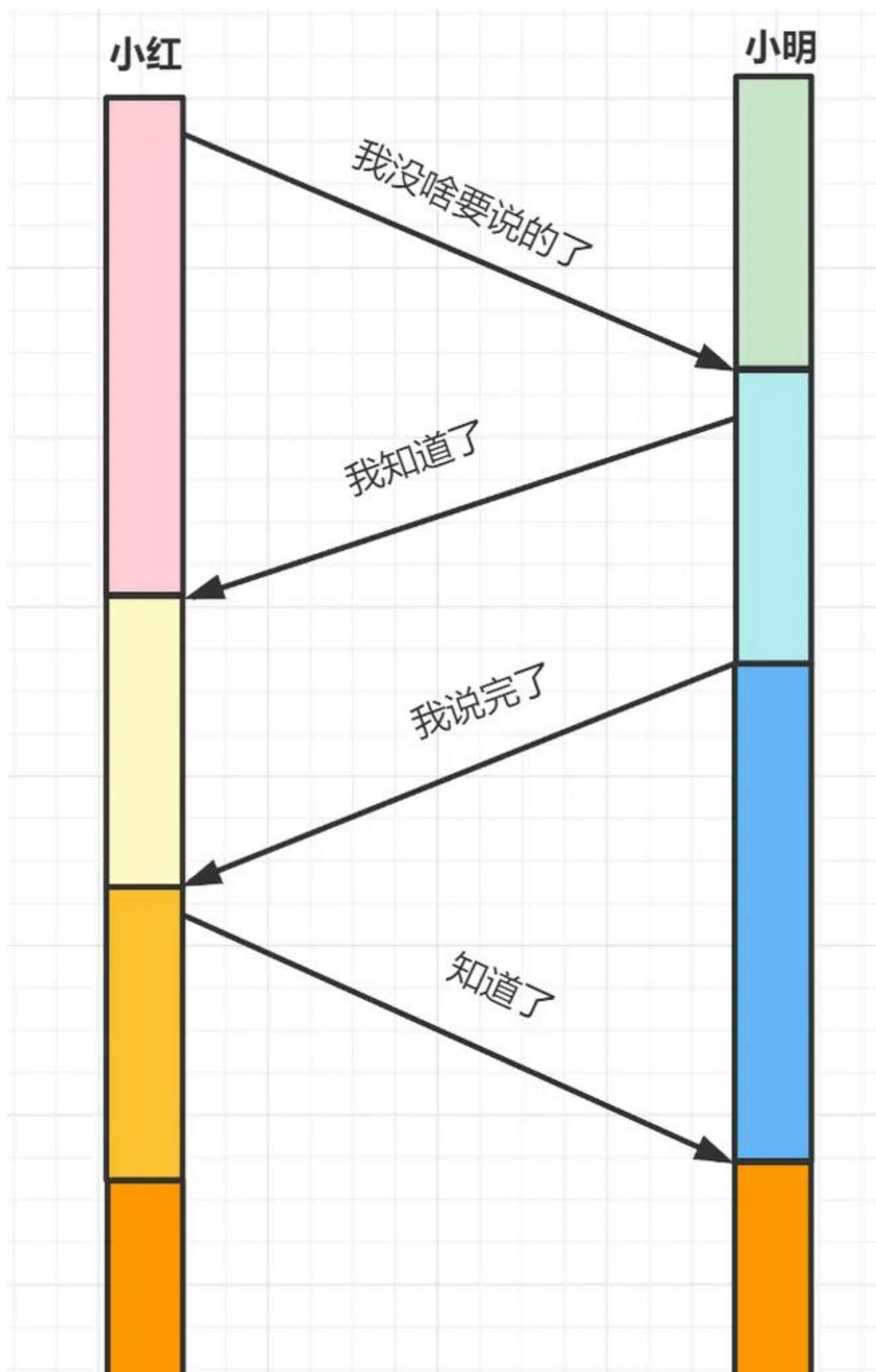
小明

我没啥要说的了

我知道了

我说完了

知道了



## 19. Session 和 Cookie 的区别。

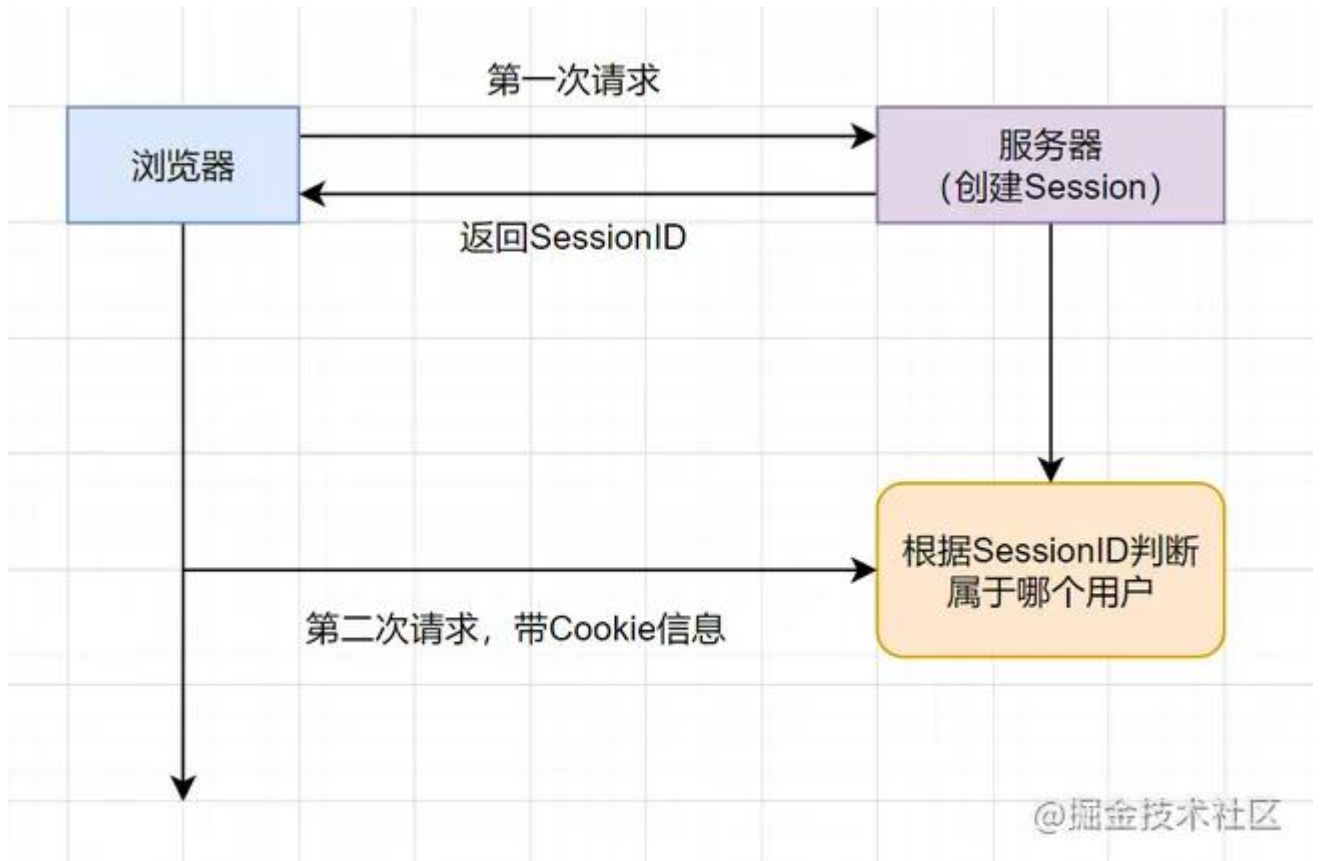
我们先来看 Session 和 Cookie 的定义：

- Cookie 是服务器发送到用户浏览器，并保存在浏览器本地的一小块文本串数据。它会在浏览器下次向同一服务器再发起请求时，被携带发送到服务器。通常，它用于告知服务端两个请求是否来自同一浏览器，一样用于保持用户的登录状态等。Cookie 使基于无状态的 HTTP 协议记录稳定的状态信息成为了可能。
- session 指的就是服务器和客户端一次会话的过程。Session 利用 Cookie 进行信息处理的，当用户首先进行了请求后，服务端就在用户浏览器上创建了一个 Cookie，当这个 Session 结束时，其实就是意味着这个 Cookie 就过期了。Session 对象存储着特定用户会话所需的属性及配置信息。

Session 和 Cookie 到底有什么不同呢？

- 存储位置不一样，Cookie 保存在客户端，Session 保存在服务器端。
- 存储数据类型不一样，Cookie 只能保存 ASCII，Session 可以存任意数据类型，一般情况下我们可以在 Session 中保持一些常用变量信息，比如说 UserId 等。
- 有效期不同，Cookie 可设置为长时间保持，比如我们经常使用的默认登录功能，Session 一般有效时间较短，客户端关闭或者 Session 超时都会失效。
- 隐私策略不同，Cookie 存储在客户端，比较容易遭到不法获取，早期有人将用户的登录名和密码存储在 Cookie 中导致信息被窃取；Session 存储在服务端，安全性相对 Cookie 要好一些。
- 存储大小不同，单个 Cookie 保存的数据不能超过 4K，Session 可存储数据远高于 Cookie。

来看个图吧：



- 用户第一次请求服务器时，服务器根据用户提交的信息，创建对应的 Session ，请求返回时将此 Session 的唯一标识信息 SessionID 返回给浏览器，浏览器接收到服务器返回的 SessionID 信息后，会将此信息存入 Cookie 中，同时 Cookie 记录此 SessionID 属于哪个域名。
- 当用户第二次访问服务器时，请求会自动判断此域名下是否存在 Cookie 信息，如果存在，则自动将 Cookie 信息也发送给服务端，服务端会从 Cookie 中获取 SessionID，再根据 SessionID 查找对应的 Session 信息，如果没有找到说明用户没有登录或者登录失效，如果找到 Session 证明用户已经登录可执行后面操作。

## 20. TCP 是如何保证可靠性的



# TCP老哥，稳！

@掘金技术社区

- 首先，TCP 的连接是基于**三次握手**，而断开则是**四次挥手**。确保连接和断开的可靠性。
- 其次，TCP 的可靠性，还体现在**有状态**；TCP 会记录哪些数据发送了，哪些数据被接受了，哪些没有被接受，并且保证数据包按序到达，保证数据传输不出差错。
- 再次，TCP 的可靠性，还体现在**可控制**。它有数据包校验、ACK 应答、**超时重传(发送方)**、失序数据重传（接收方）、丢弃重复数据、流量控制（滑动窗口）和拥塞控制等机制。

## 21. TCP 和 UDP 的区别

1. TCP 面向连接（（如打电话要先拨号建立连接）;UDP 是无连接的，即发送数据之前不需要建立连接。
2. TCP 要求安全性，提供可靠的服务，通过 TCP 连接传送的数据，不丢失、不重复、安全可靠。而 UDP 尽最大努力交付，即不保证可靠交付。
3. TCP 是点对点连接的，UDP 一对一，一对多，多对多都可以
4. TCP 传输效率相对较低,而 UDP 传输效率高，它适用于对高速传输和实时性有较高的通信或广播通信。
5. TCP 适合用于网页，邮件等;UDP 适合用于视频，语音广播等
6. TCP 面向字节流，UDP 面向报文

## 22. TCP 报文首部有哪些字段，说说其作用

|          |      |  |     |     |     |     |          |     |               |  |  |  |  |
|----------|------|--|-----|-----|-----|-----|----------|-----|---------------|--|--|--|--|
| 16位源端口号  |      |  |     |     |     |     | 16位目的端口号 |     |               |  |  |  |  |
| 32位序号    |      |  |     |     |     |     |          |     |               |  |  |  |  |
| 32位确认号   |      |  |     |     |     |     |          |     |               |  |  |  |  |
| 4位头部长度   | 6位保留 |  | URG | ACK | PSH | RST | SYN      | FIN | 16位窗口大小       |  |  |  |  |
| 16位校验和   |      |  |     |     |     |     |          |     | 16位紧急指针       |  |  |  |  |
| 选项（长度可变） |      |  |     |     |     |     |          |     | 填充<br>@掘金技术社区 |  |  |  |  |

- **16 位端口号：**源端口号，主机该报文段是来自哪里；目标端口号，要传给哪个上层协议或应用程序
- **32 位序号：**一次 TCP 通信（从 TCP 连接建立到断开）过程中某一个传输方向上的字节流的每个字节的编号。
- **32 位确认号：**用作对另一方发送的 tcp 报文段的响应。其值是收到的 TCP 报文段的序号值加 1。
- **4 位头部长度：**表示 tcp 头部有多少个 32bit 字（4 字节）。因为 4 位最大能标识 15，所以 TCP 头部最长是 60 字节。
- **6 位标志位：**URG(紧急指针是否有效)，ACK（表示确认号是否有效），PSH（缓冲区尚未填满），RST（表示要求对方重新建立连接），SYN（建立连接消息标志接），FIN（表示告知对方本端要关闭连接了）
- **16 位窗口大小：**是 TCP 流量控制的一个手段。这里说的窗口，指的是接收通告窗口。它告诉对方本端的 TCP 接收缓冲区还能容纳多少字节的数据，这样对方就可以控制发送数据的速度。
- **16 位校验和：**由发送端填充，接收端对 TCP 报文段执行 CRC 算法以检验 TCP 报文段在传输过程中是否损坏。注意，这个校验不仅包括 TCP 头部，也包括数据部分。这也是 TCP 可靠传输的一个重要保障。

- **16 位紧急指针：**一个正的偏移量。它和序号字段的值相加表示最后一个紧急数据的下一字节的序号。因此，确切地说，这个字段是紧急指针相对当前序号的偏移，不妨称之为紧急偏移。TCP 的紧急指针是发送端向接收端发送紧急数据的方法。

## 23. HTTP 状态码 301 和 302 的区别？

- **301（永久移动）**请求的网页已被永久移动到新位置。服务器返回此响应（作为对 GET 或 HEAD 请求的响应）时，会自动将请求者转到新位置。
- **302：（临时移动）**服务器目前正从不同位置的网页响应请求，但请求者应继续使用原有位置来进行以后的请求。此代码与响应 GET 和 HEAD 请求的 301 代码类似，会自动将请求者转到不同的位置。

HTTP 状态码 301 与 302 的区别：

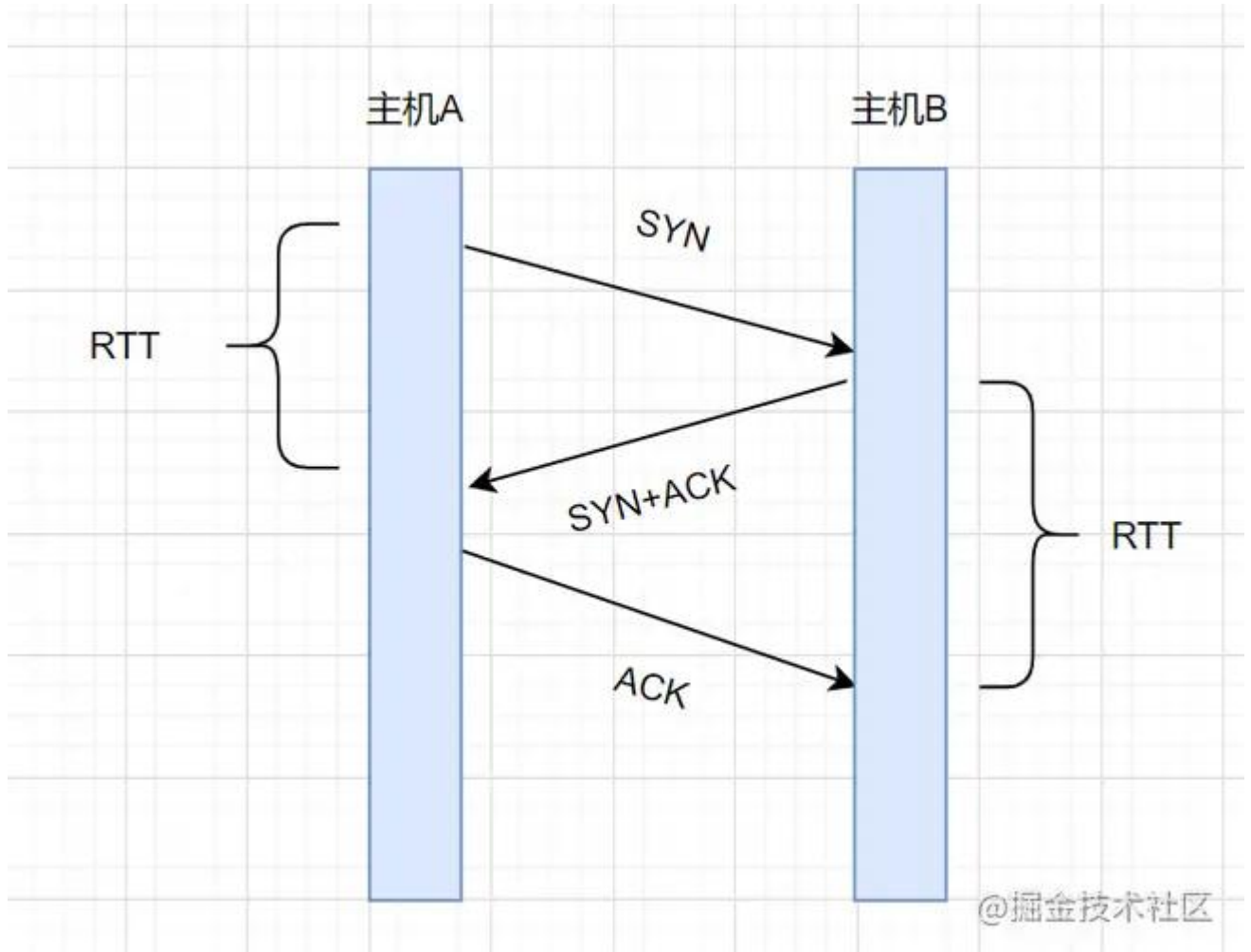
1. 它们之间关键区别在，资源是否存在有效性；
2. 301 资源还在只是换了一个位置，返回的是新位置的内容；
3. 302 资源暂时失效，返回的是一个临时的代替页上。

## 24. 聊聊 TCP 的重传机制

### 超时重传

TCP 为了实现可靠传输，实现了重传机制。最基本的重传机制，就是**超时重传**，即在发送数据报文时，设定一个定时器，每间隔一段时间，没有收到对方的 ACK 确认应答报文，就会重发该报文。

这个间隔时间，一般设置为多少呢？我们先来看下什么叫 **RTT（Round-Trip Time，往返时间）**。



RTT 就是，一个数据包从发出去到回来的时间，即数据包的一次往返时间。超时重传时间，就是 Retransmission Timeout ，简称 **RTO**。

### RTO 设置多久呢？

- 如果 RTO 比较小，那很可能数据都没有丢失，就重发了，这会导致网络阻塞，会导致更多的超时出现。
- 如果 RTO 比较大，等到花儿都谢了还是没有重发，那效果就不好了。

一般情况下，RTO 略大于 RTT，效果是最好的。一些小伙伴会问，超时时间有没有计算公式呢？有的！有个标准方法算 RTO 的公式，也叫 **Jacobson / Karels 算法**。我们一起来看下计算 RTO 的公式

#### 1. 先计算 SRTT（计算平滑的 RTT）

$$SRTT = (1 - \alpha) * SRTT + \alpha * RTT$$
 //求 SRTT 的加权平均复制代码

#### 2. 再计算 RTTVAR (round-trip time variation)

$RTTVAR = (1 - \beta) * RTTVAR + \beta * (|RTT - SRTT|)$  //计算 SRTT 与真实值的差距  
复制代码

### 3. 最终的 RTO

$RTO = \mu * SRTT + \theta * RTTVAR = SRTT + 4 * RTTVAR$  复制代码

其中， $\alpha = 0.125$ ， $\beta = 0.25$ ， $\mu = 1$ ， $\theta = 4$ ，这些参数都是大量结果得出的最优参数。

但是，超时重传会有这些缺点：

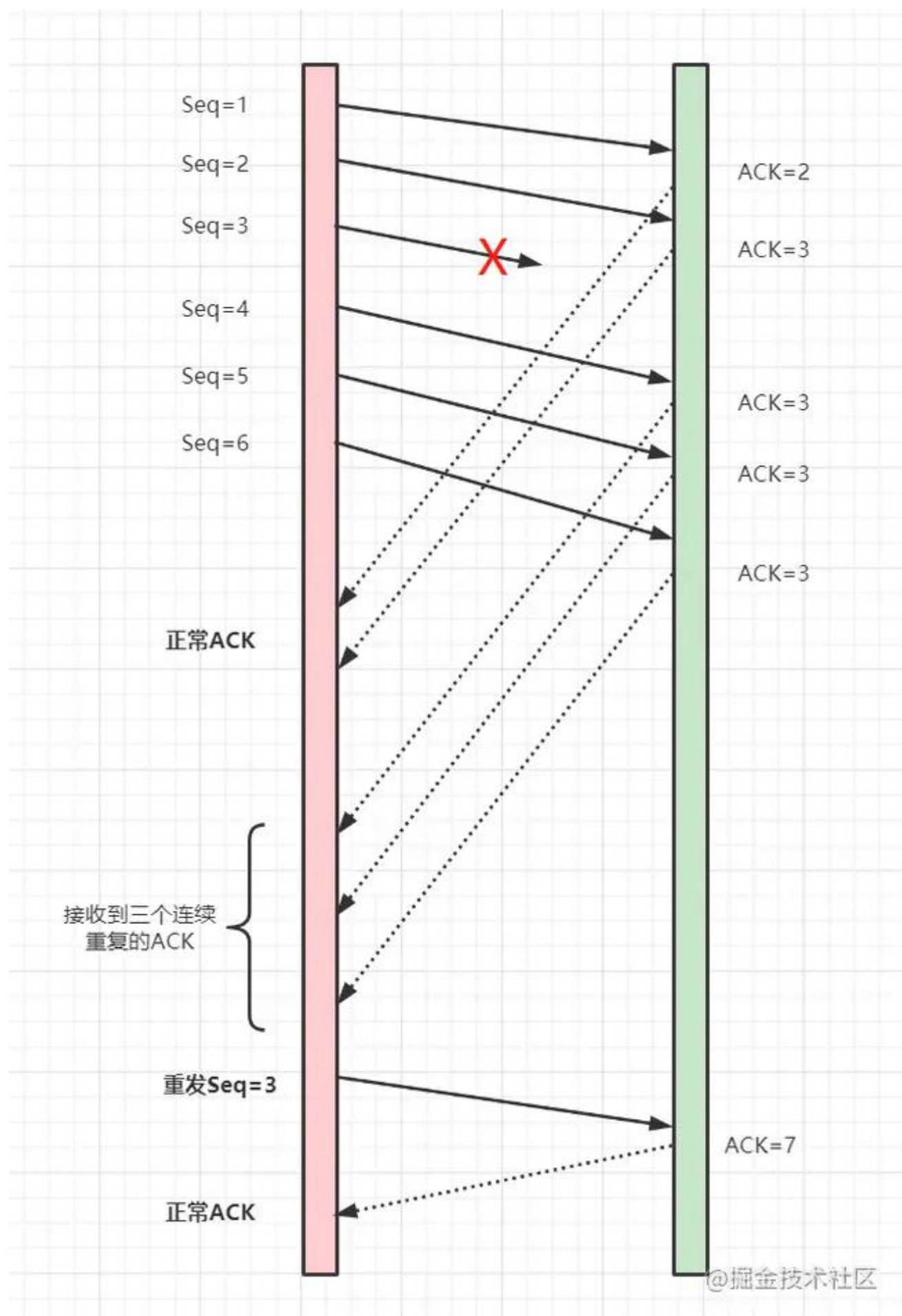
- 当一个报文段丢失时，会等待一定的超时周期然后才重传分组，增加了端到端的时延。
- 当一个报文段丢失时，在其等待超时的过程中，可能会出现这种情况：其后的报文段已经被接收端接收但却迟迟得不到确认，发送端会认为也丢失了，从而引起不必要的重传，既浪费资源也浪费时间。

并且，TCP 有个策略，就是超时时间间隔会加倍。超时重传需要等待很长时间。因此，还可以使用**快速重传**机制。

#### 快速重传

**快速重传**机制，它不以时间驱动，而是以数据驱动。它基于接收端的反馈信息来引发重传。

一起来看下快速重传流程：



发送端发送了 1, 2, 3, 4, 5, 6 份数据:

- 第一份 Seq=1 先送到了, 于是就 Ack 回 2;
- 第二份 Seq=2 也送到了, 假设也正常, 于是 ACK 回 3;
- 第三份 Seq=3 由于网络等其他原因, 没送到;
- 第四份 Seq=4 也送到了, 但是因为 Seq3 没收到。所以 ACK 回 3;
- 后面的 Seq=4,5 的也送到了, 但是 ACK 还是回复 3, 因为 Seq=3 没收到。
- 发送端连着收到三个重复冗余 ACK=3 的确认 (实际上是 4 个, 但是前面一个是正常的 ACK, 后面三个才是重复冗余的), 便知道哪个报文段在传输过程中丢失了, 于是在定时器过期之前, 重传该报文段。
- 最后, 接收到收到了 Seq3, 此时因为 Seq=4, 5, 6 都收到了, 于是 ACK 回 7.

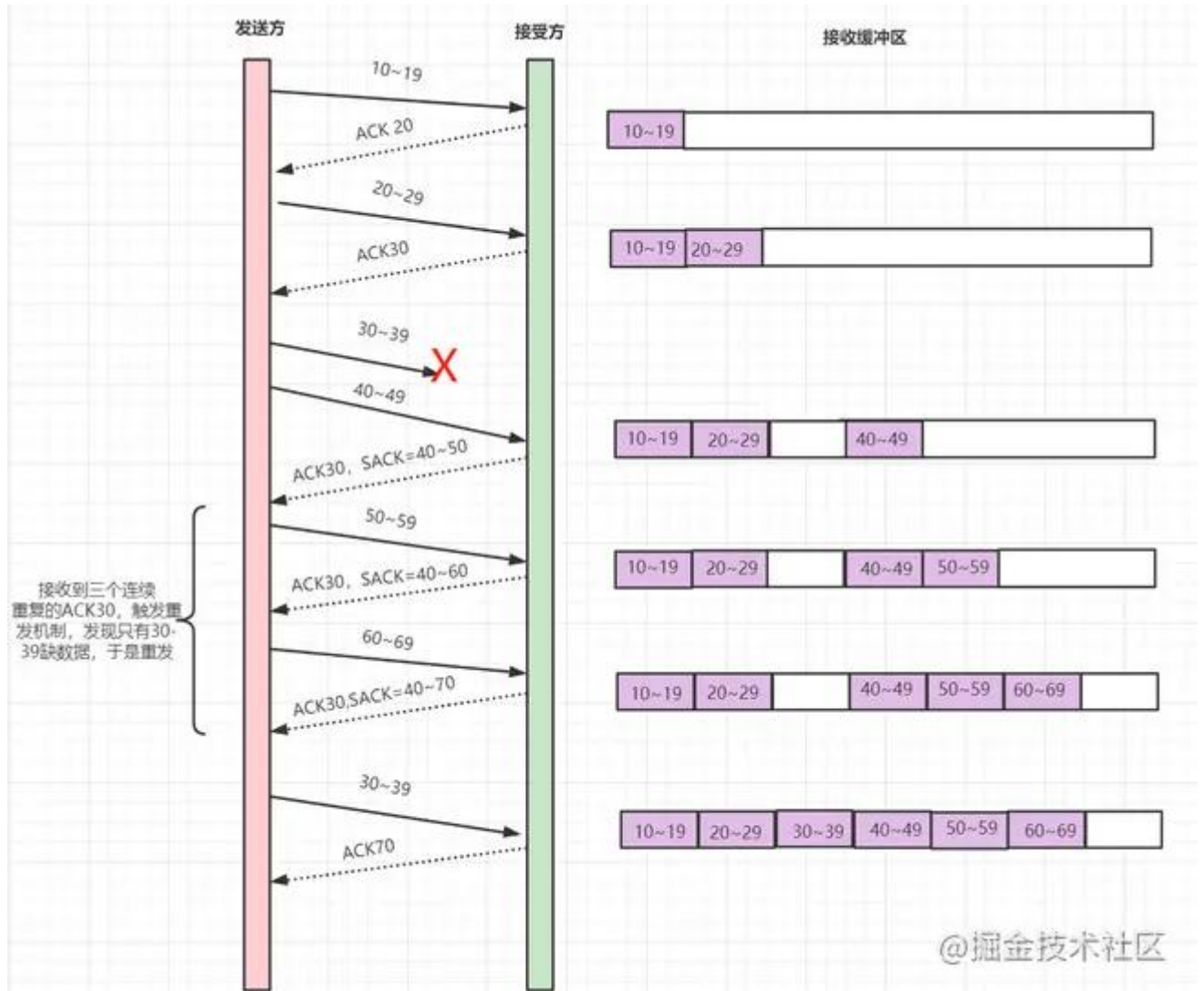
但**快速重传**还可能会有个问题: ACK 只向发送端告知最大的有序报文段, 到底是哪个报文丢失了呢? **并不确定**! 那到底该重传多少个包呢?

是重传 Seq3 呢? 还是重传 Seq3、Seq4、Seq5、Seq6 呢? 因为发送端并不清楚这三个连续的 ACK3 是谁传回来的。

### 带选择确认的重传 (SACK)

为了解决快速重传的问题: **应该重传多少个包?** TCP 提供了 **SACK 方法** (带选择确认的重传, Selective Acknowledgment)。

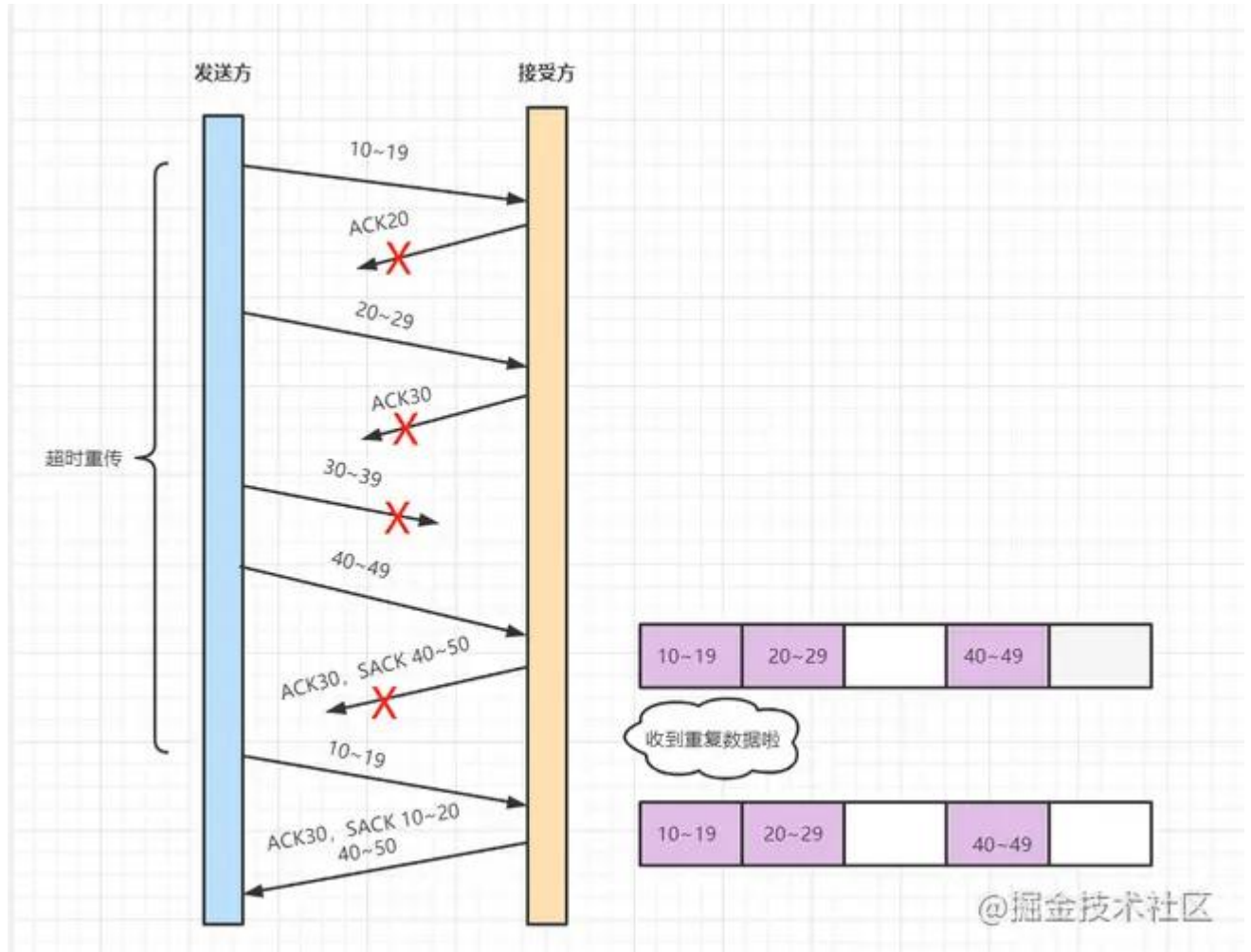
**SACK 机制**就是, 在快速重传的基础上, 接收端返回最近收到的报文段的序列号范围, 这样发送端就知道接收端哪些数据包没收到, 酱紫就很清楚该重传哪些数据包啦。SACK 标记是加在 TCP 头部**选项**字段里面的。



如上图，发送端收到了三次同样的  $ACK=30$  的确认报文，于是就会触发快速重发机制，通过  $SACK$  信息发现只有  $30\sim39$  这段数据丢失，于是重发时就只选择了这个  $30\sim39$  的 TCP 报文段进行重发。

## D-SACK

D-SACK，即 Duplicate SACK（重复 SACK），在 SACK 的基础上做了一些扩展，，主要用来告诉发送方，有哪些数据包自己重复接受了。DSACK 的目的是帮助发送方判断，是否发生了包失序、ACK 丢失、包重复或伪重传。让 TCP 可以更好的做网络流控。来看个图吧：



## 25. IP 地址有哪些分类？

一句话概括，IP 地址 = 网络号+主机号。

1. 网络号：它标志主机（或路由器）所连接到的网络，网络地址表示属于互联网的哪一个网络
2. 主机号：它标志该主机（或路由器），主机地址表示其属于该网络中的哪一台主机

IP 地址 分为 A，B，C，D，E 五大类：

- A 类地址(1~126)：以 0 开头，网络号占前 8 位，主机号占后 24 位。
- B 类地址(128~191)：以 10 开头，网络号占前 16 位，主机号占后 16 位。
- C 类地址(192~223)：以 110 开头，网络号占前 24 位，主机号占后 8 位。
- D 类地址(224~239)：以 1110 开头，保留位多播地址。
- E 类地址(240~255)：以 11110 开头，保留位为将来使用

|       |       |            |           |                           |
|-------|-------|------------|-----------|---------------------------|
| A类地址: | 0     | 网络号 (7位)   | 主机号 (24位) | 1.0.0.1-127.255.255.254   |
| B类地址: | 10    | 网络号 (14位)  | 主机号 (16位) | 128.0.0.1-191.255.255.254 |
| C类地址: | 110   | 网络号 (21位)  | 主机号 (8位)  | 192.0.0.1-223.255.255.254 |
| D类地址: | 1110  | 组播地址 (28位) |           | 224.0.0.1~239.255.255.254 |
| E类地址: | 11110 | 保留为将来使用    |           | 240.0.0.1~255.255.255.254 |

@掘金技术社区

## 26. 聊聊 TCP 的滑动窗口

TCP 发送一个数据，需要收到确认应答，才会发送下一个数据。这样有个缺点，就是效率会比较低。

这就好像我们面对面聊天，你说完一句，我应答后，你才会说下一句。那么，如果我在忙其他事情，没有能够及时回复你。你说完一句后，要等到我忙完回复你，你才说下句，这显然很不现实。

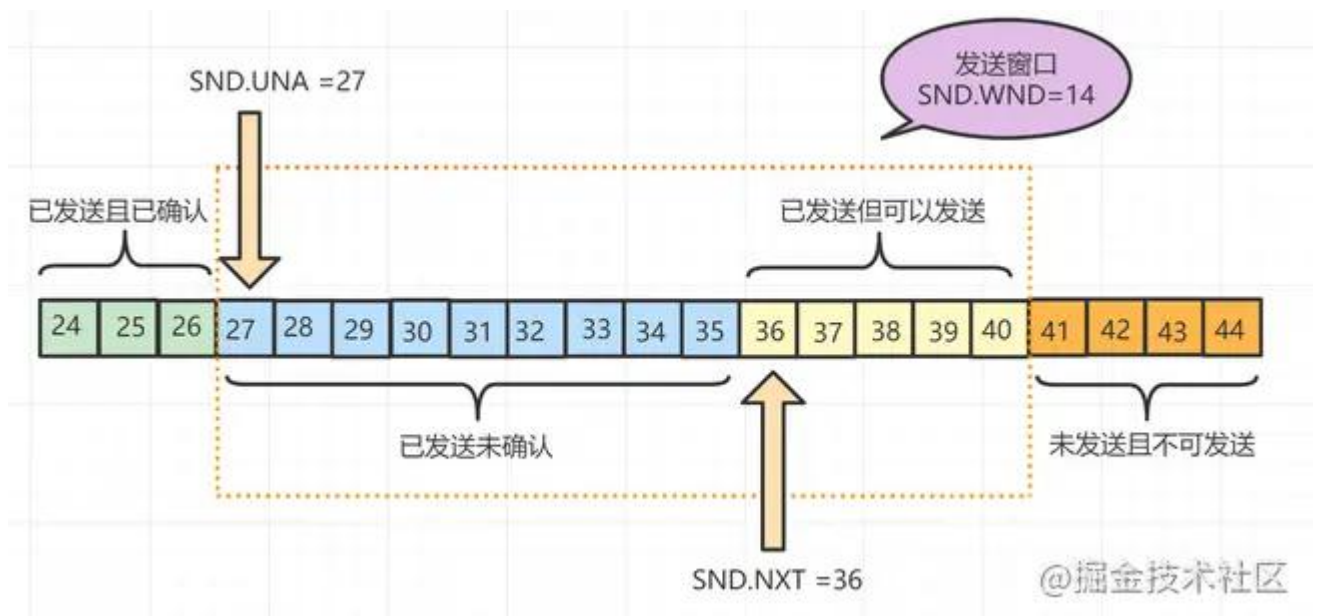
为了解决这个问题，TCP 引入了**窗口**，它是操作系统开辟的一个缓存空间。窗口大小值表示无需等待确认应答，而可以继续发送数据的最大值。

TCP 头部有个字段叫 **win**，也即那个 **16 位的窗口大小**，它告诉对方本端的 TCP 接收缓冲区还能容纳多少字节的数据，这样对方就可以控制发送数据的速度，从而达到**流量控制**的目的。

通俗点讲，就是接受方每次收到数据包，在发送确认报文的时候，同时告诉发送方，自己的缓存区还有多少空余空间，缓冲区的空余空间，我们就称之为接受窗口大小。这就是 win。

TCP 滑动窗口分为两种：发送窗口和接收窗口。**发送端的滑动窗口**包含四大部分，如下：

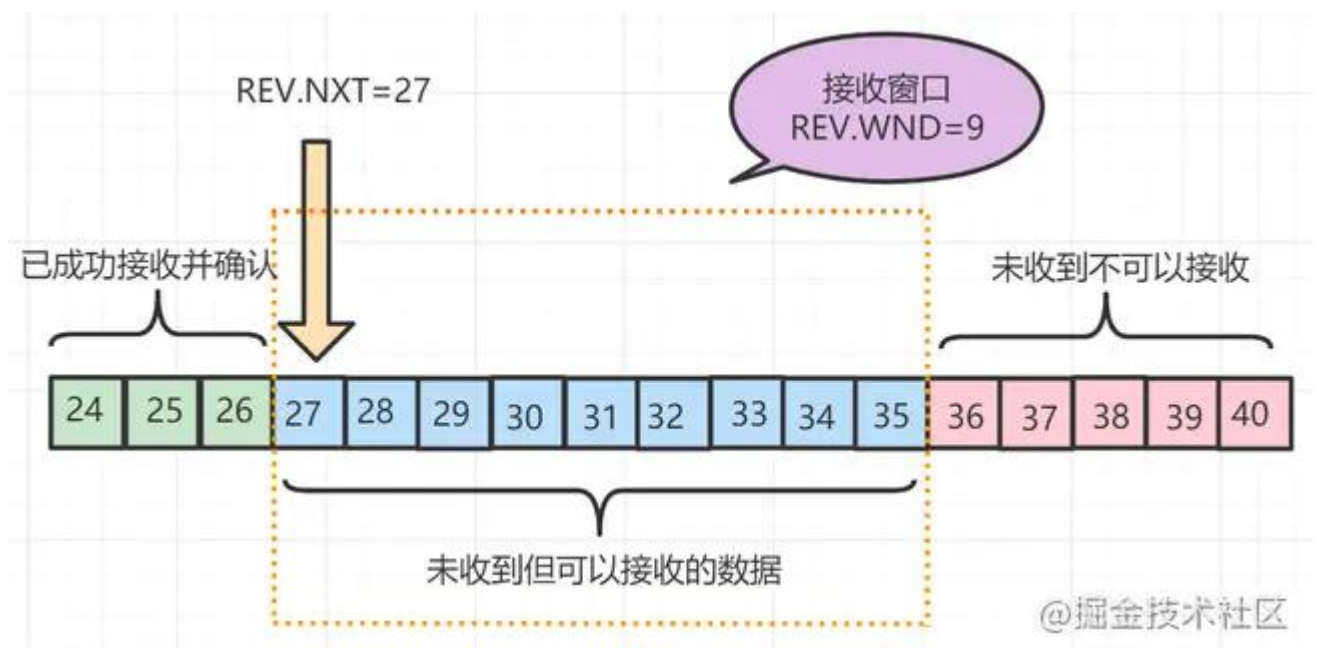
- 已发送且已收到 ACK 确认
- 已发送但未收到 ACK 确认
- 未发送但可以发送
- 未发送也不可以发送



- 虚线矩形框，就是发送窗口。
- $SND.WND$ : 表示发送窗口的大小,上图虚线框的格子数就是 14 个。
- $SND.UNA$ : 一个绝对指针，它指向的是已发送但未确认的第一个字节的序列号。
- $SND.NXT$ : 下一个发送的位置，它指向未发送但可以发送的第一个字节的序列号。

接收方的滑动窗口包含三大部分，如下：

- 已成功接收并确认
- 未收到数据但可以接收
- 未收到数据并不可以接收的数据



- 虚线矩形框，就是接收窗口。

- REV.WND: 表示接收窗口的大小,上图虚线框的格子就是 9 个。
- REV.NXT:下一个接收的位置,它指向未收到但可以接收的第一个字节的序列号。

27. 聊聊五层计算机网络体系结构中,每一层对应的网络协议有哪些?

# 网络协议

## 应用层

- HTTP 超文本传输协议
- FTP 文本传输协议
- SMTP 简单邮件传输协议
- DNS 域名系统
- SSH 安全外壳协议
- DHCP 动态主机配置协议
- TELNET 远程登录协议

## 传输层

- TCP 传输控制协议
- UDP 用户数据报文协议

## 网络层

- IP 网际协议
- ARP 地址转换协议
- RARP 反向地址转换协议
- ICMP Internet控制报文协议
- IGMP Internet 组管理协议
- RIP 路由信息协议
- OSPF 分布式链路状态协议
- BGP 边界网关协议

## 数据链路层

- ARQ 自动重传请求协议
- CSMA/CD 停止等待协议
- PPP 点对点协议

## 物理层

- HUB
- 网线
- 中继器

## 28. 聊聊 TCP 的流量控制

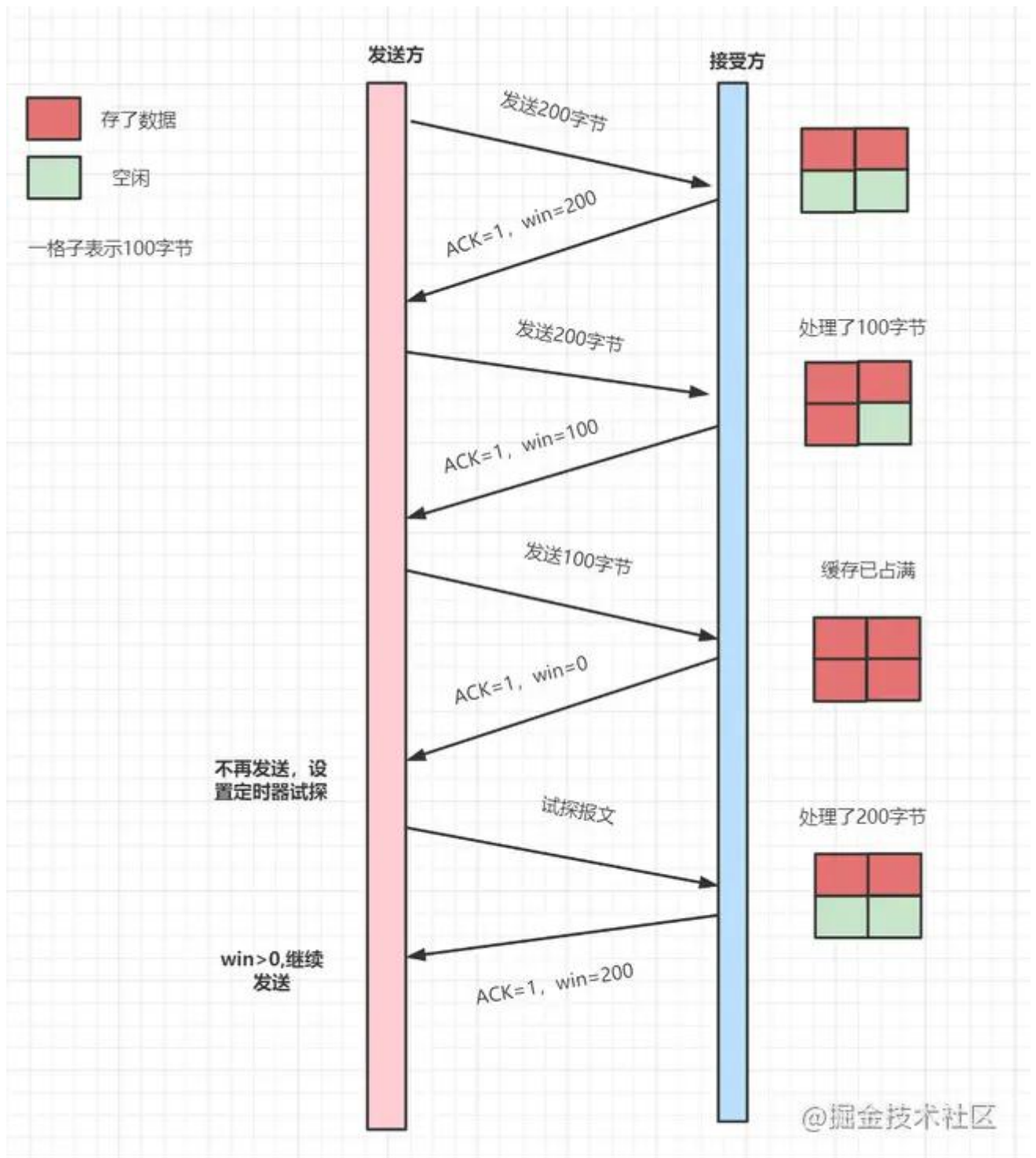
TCP 三次握手，发送端和接收端进入到 ESTABLISHED 状态，它们即可以愉快地传输数据啦。

但是发送端不能疯狂地向接收端发送数据，因为接收端接收不过来的话，接收方只能把处理不过来的数据存在缓存区里。如果缓存区都满了，发送方还在疯狂发送数据的话，接收方只能把收到的数据包丢掉，这就浪费了网络资源啦。

TCP 提供一种机制可以让发送端根据接收端的实际接收能力控制发送的数据量，这就是**流量控制**。

TCP 通过滑动窗口来控制流量，我们看下流量控制的**简要流程**吧：

首先双方三次握手，初始化各自的窗口大小，均为 4096 个字节。



1. 假如当前发送方给接收方发送了 200 个字节, 那么, 发送方的 `SND.NXT` 会右移 200 个字节, 也就是说当前的可用窗口减少了 200 个字节。
2. 接受方收到后, 放到缓冲队列里面,  $REV.WND = 400 - 200 = 200$  字节, 所以 `win=200` 字节返回给发送方。接收方会在 `ACK` 的报文首部带上缩小后的滑动窗口 200 字节
3. 发送方又发送 200 字节过来, 200 字节到达, 继续放到缓冲队列。不过这时候, 由于大量负载的原因, 接受方处理不了这么多字节, 只能处理 100 字节, 剩余的 100

字节继续放到缓冲队列。这时候， $REV.WND = 400 - 200 - 100 = 100$  字节，即  $win=100$  返回发送方。

4. 发送方继续干活，发送 100 字节过来，这时候，接受窗口  $win$  变为 0。
5. 发送方停止发送，开启一个定时任务，每隔一段时间，就去询问接受方，直到  $win$  大于 0，才继续开始发送。

## 29. 说下 ARP 协议的工作原理？

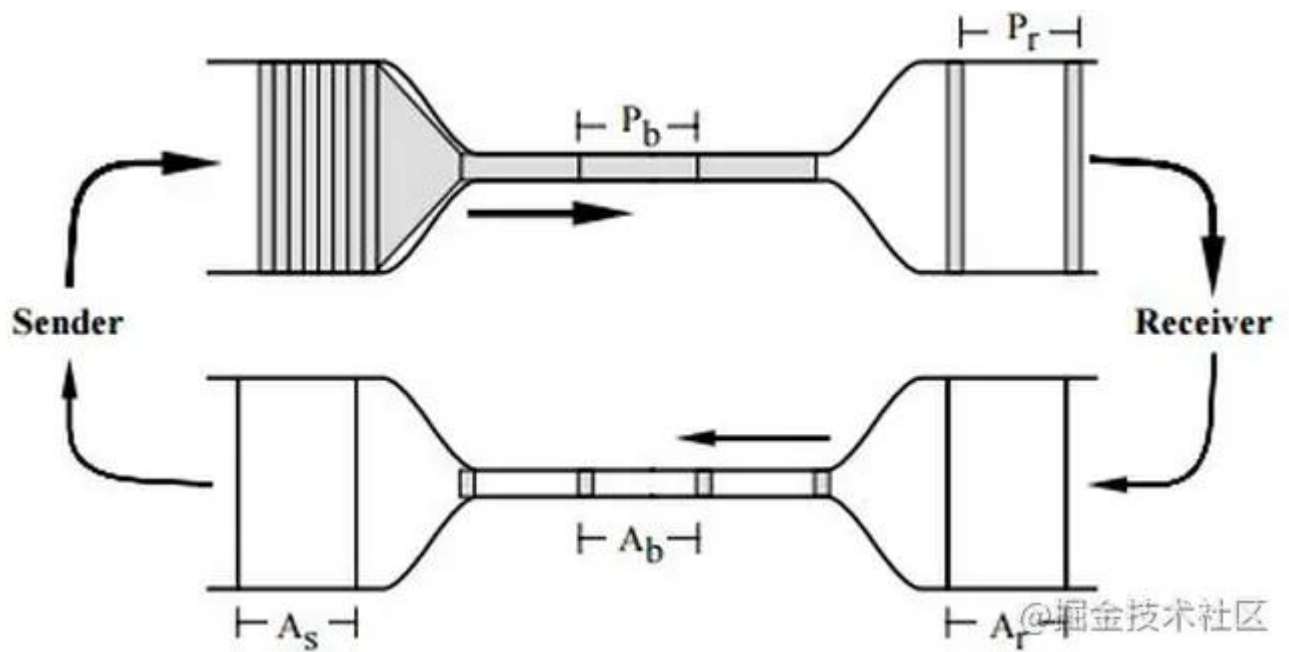
ARP 协议，即 **Address Resolution Protocol**，地址解析协议，用于实现 IP 地址到 MAC 地址的映射。

1. 首先，每台主机都会在自己的 ARP 缓冲区中建立一个 ARP 列表，以表示 IP 地址和 MAC 地址的对应关系。
2. 当源主机需要将一个数据包发送到目的主机时，会首先检查自己的 ARP 列表，是否存在该 IP 地址对应的 MAC 地址；如果存在，就直接将数据包发送到这个 MAC 地址；如果不存在，就向本地网段发起一个 ARP 请求的广播包，查询此目的主机对应的 MAC 地址。此 ARP 请求的数据包里，包括源主机的 IP 地址、硬件地址、以及目的主机的 IP 地址。
3. 网络中的所有的主机收到这个 ARP 请求后，会检查数据包中的目的 IP 是否和自己的 IP 地址一致。如果不相同就忽略此数据包；如果相同，该主机首先将发送端的 MAC 地址和 IP 地址添加到自己的 ARP 列表中，如果 ARP 表中已经存在该 IP 的信息，则将其覆盖，然后给源主机发送一个 ARP 响应数据包，告诉对方自己是它需要查找的 MAC 地址。
4. 源主机收到这个 ARP 响应数据包后，将得到的目的主机的 IP 地址和 MAC 地址添加到自己的 ARP 列表中，并利用此信息开始数据的传输。如果源主机一直没有收到 ARP 响应数据包，表示 ARP 查询失败。

## 30. 说下 TCP 的拥塞控制

拥塞控制是作用于网络的，防止过多的数据包注入到网络中，避免出现网络负载过大的情况。它的目标主要是最大化利用网络上瓶颈链路的带宽。它跟流量控制又有什么区别呢？流量控制是作用于接收者的，根据接收端的实际接收能力控制发送速度，防止分组丢失的。

我们可以把网络链路比喻成一根水管，如果我们想最大化利用网络来传输数据，那就是尽快让水管达到最佳充满状态。



发送方维护一个**拥塞窗口 cwnd (congestion window)** 的变量，用来估算在一段时间内这条链路（水管）可以承载和运输的数据（水）的数量。它大小代表着网络的拥塞程度，并且是动态变化的，但是为了达到最大的传输效率，我们该如何知道这条水管的运送效率是多少呢？

一个比较简单的方法就是不断增加传输的水量，直到水管快要爆裂为止（对应到网络上就是发生丢包），用 TCP 的描述就是：

只要网络中没有出现拥塞，拥塞窗口的值就可以再增大一些，以便把更多的数据包发送出去，但只要网络出现拥塞，拥塞窗口的值就应该减小一些，以减少注入到网络中的数据包数。

实际上，拥塞控制主要有这几种常用算法

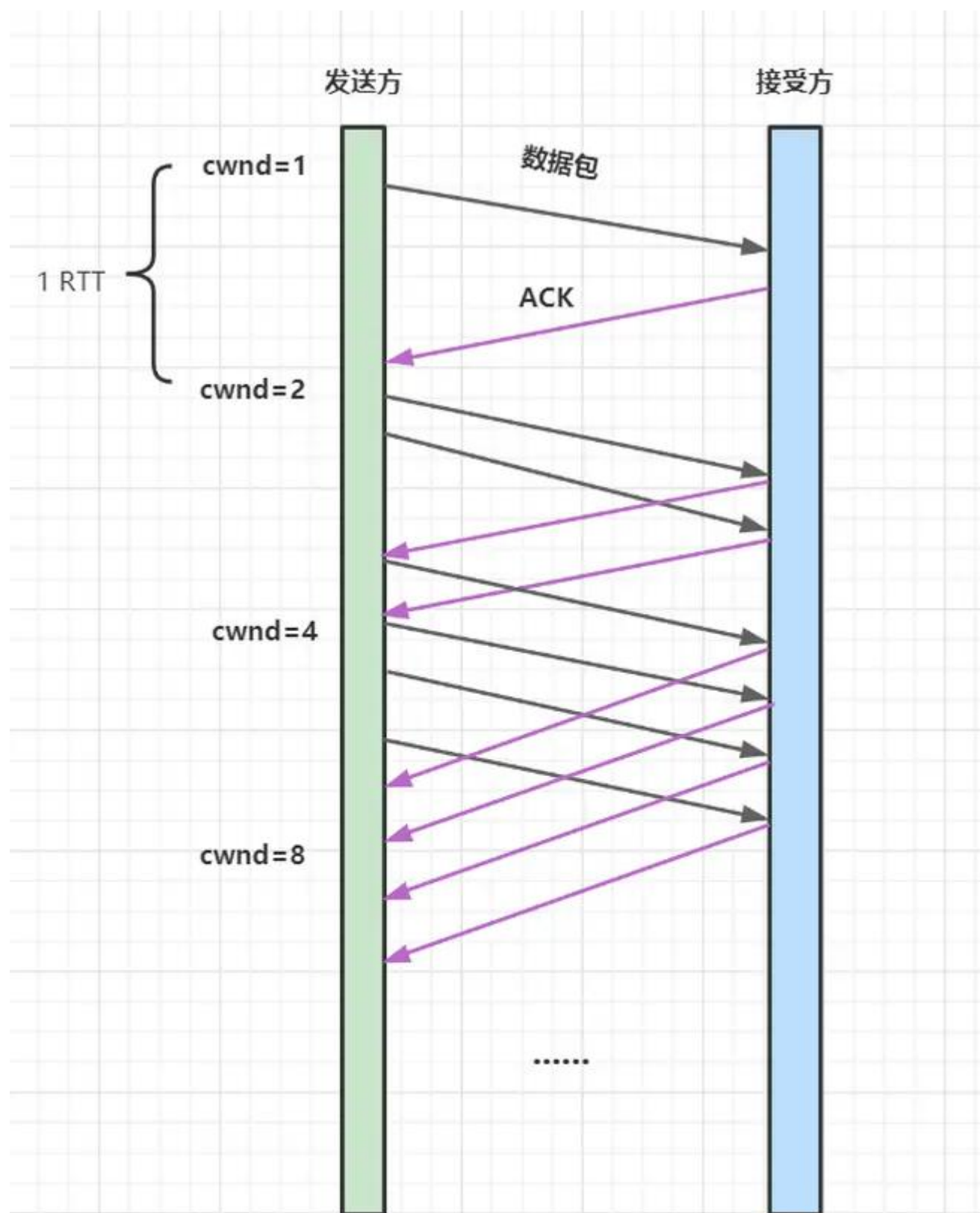
- 慢启动
- 拥塞避免
- 拥塞发生
- 快速恢复

### 慢启动算法

慢启动算法，表面意思就是，别急慢慢来。它表示 TCP 建立连接完成后，一开始不要发送大量的数据，而是先探测一下网络的拥塞程度。由小到大逐渐增加拥塞窗口的大小，如果没有出现丢包，**每收到一个 ACK，就将拥塞窗口 cwnd 大小就加 1（单位是 MSS）**。**每轮次发送窗口增加一倍，呈指数增长**，如果出现丢包，拥塞窗口就减半，进入拥塞避免阶段。

- TCP 连接完成，初始化  $cwnd = 1$ ，表明可以传一个 MSS 单位大小的数据。

- 每当收到一个 ACK, cwnd 就加一;
- 每当过了一个 RTT, cwnd 就增加一倍; 呈指数让升



每收到一个ACK，就将拥塞窗口cwnd大小就加1

每当过了一个RTT，cwnd就增加一倍

@掘金技术社区

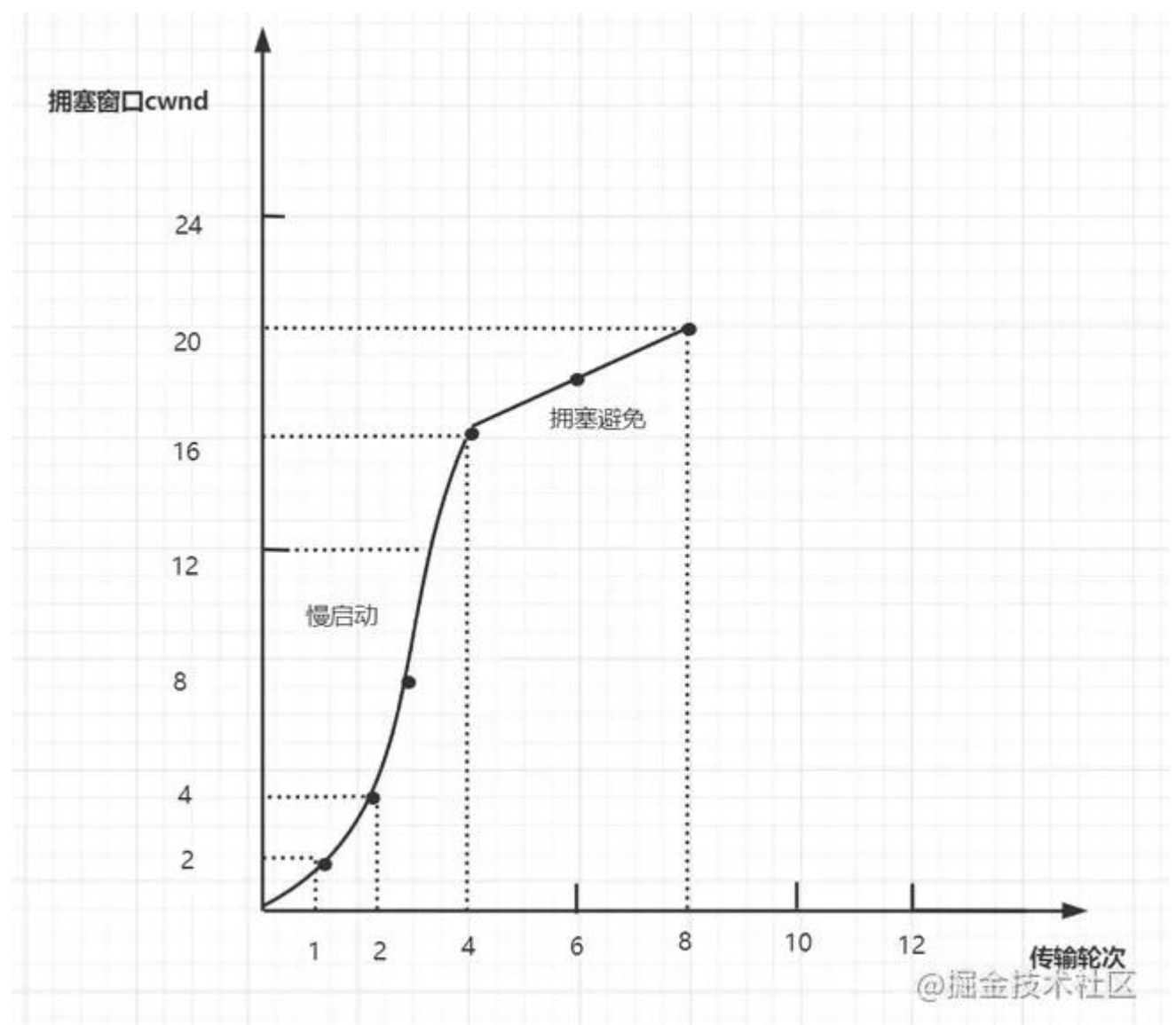
为了防止  $cwnd$  增长过大引起网络拥塞，还需设置一个慢启动阈值  $ssthresh$  (slow start threshold) 状态变量。当  $cwnd$  到达该阈值后，就好像水管被关小了水龙头一样，减少拥塞状态。即当  $cwnd > ssthresh$  时，进入了拥塞避免算法。

## 拥塞避免算法

一般来说，慢启动阈值  $ssthresh$  是 65535 字节， $cwnd$  到达慢启动阈值后

- 每收到一个 ACK 时， $cwnd = cwnd + 1/cwnd$
- 当每过一个 RTT 时， $cwnd = cwnd + 1$

显然这是一个线性上升的算法，避免过快导致网络拥塞问题。



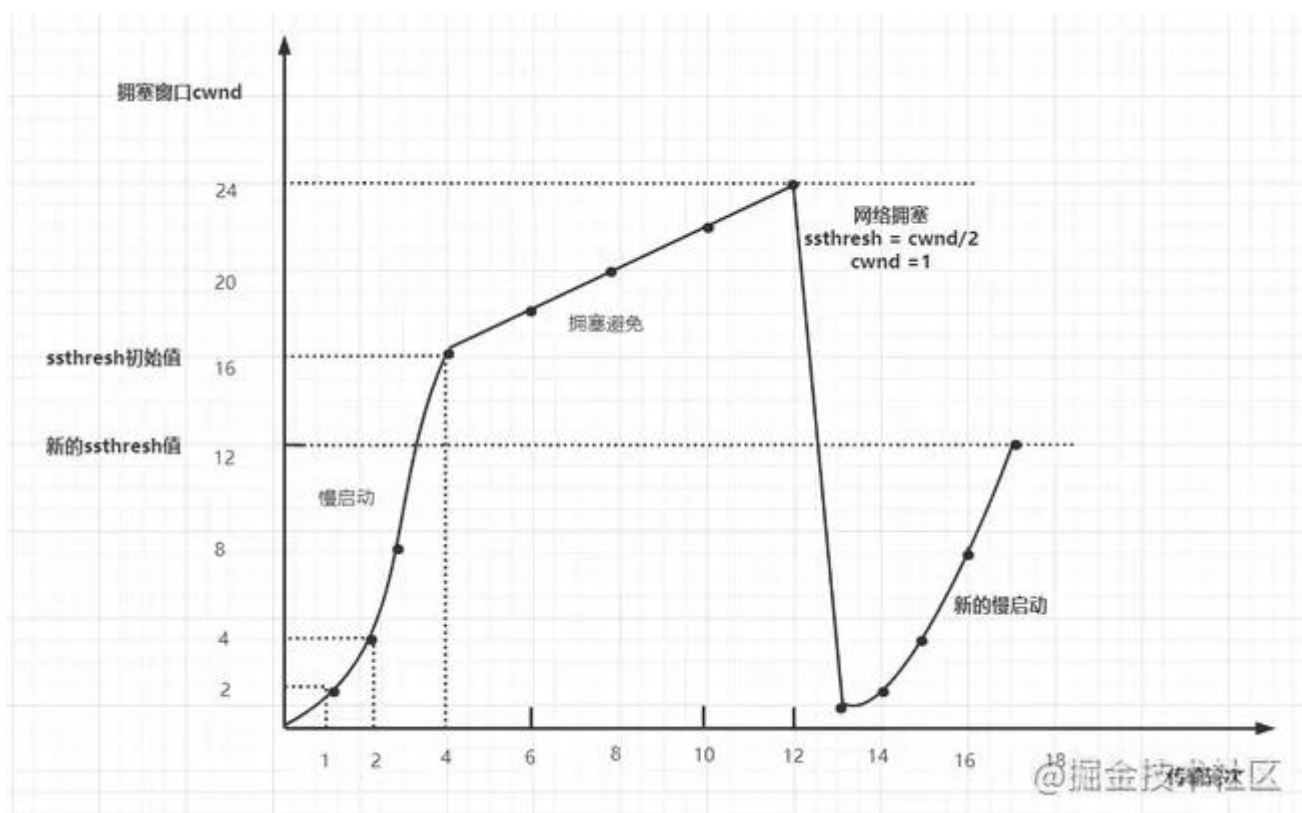
## 拥塞发生

当网络拥塞发生丢包时，会有两种情况：

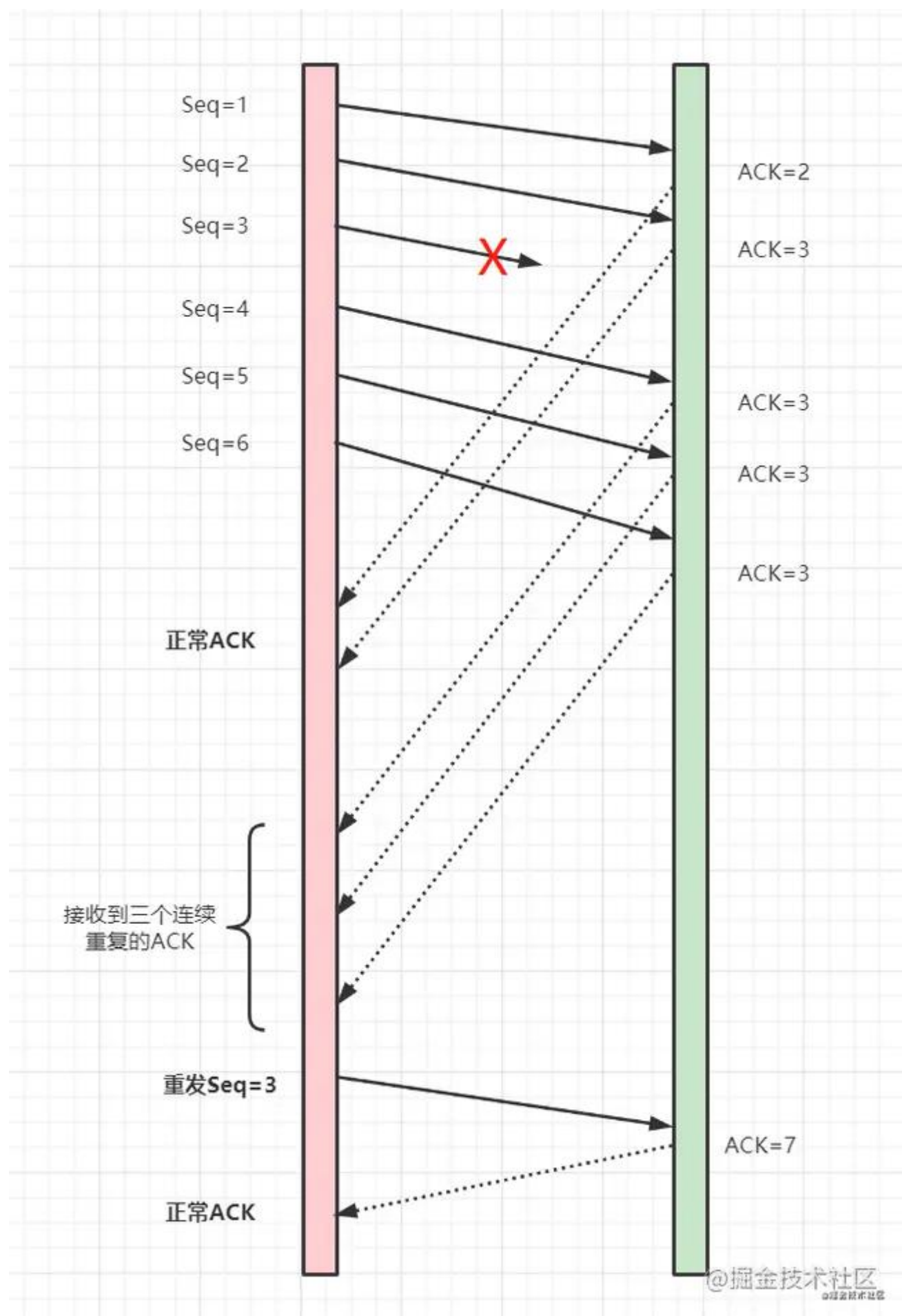
- RTO 超时重传
- 快速重传

如果是发生了 **RTO 超时重传**，就会使用拥塞发生算法

- 慢启动阈值  $ssthresh = cwnd / 2$
- $cwnd$  重置为 1
- 进入新的慢启动过程



这真的是辛辛苦苦几十年，一朝回到解放前。其实还有更好的处理方式，就是**快速重传**。发送方收到 3 个连续重复的 ACK 时，就会快速地重传，不必等待 **RTO 超时**再重传。



慢启动阈值  $ssthresh$  和  $cwnd$  变化如下：

- 拥塞窗口大小  $cwnd = cwnd / 2$
- 慢启动阈值  $ssthresh = cwnd$
- 进入快速恢复算法

## 快速恢复

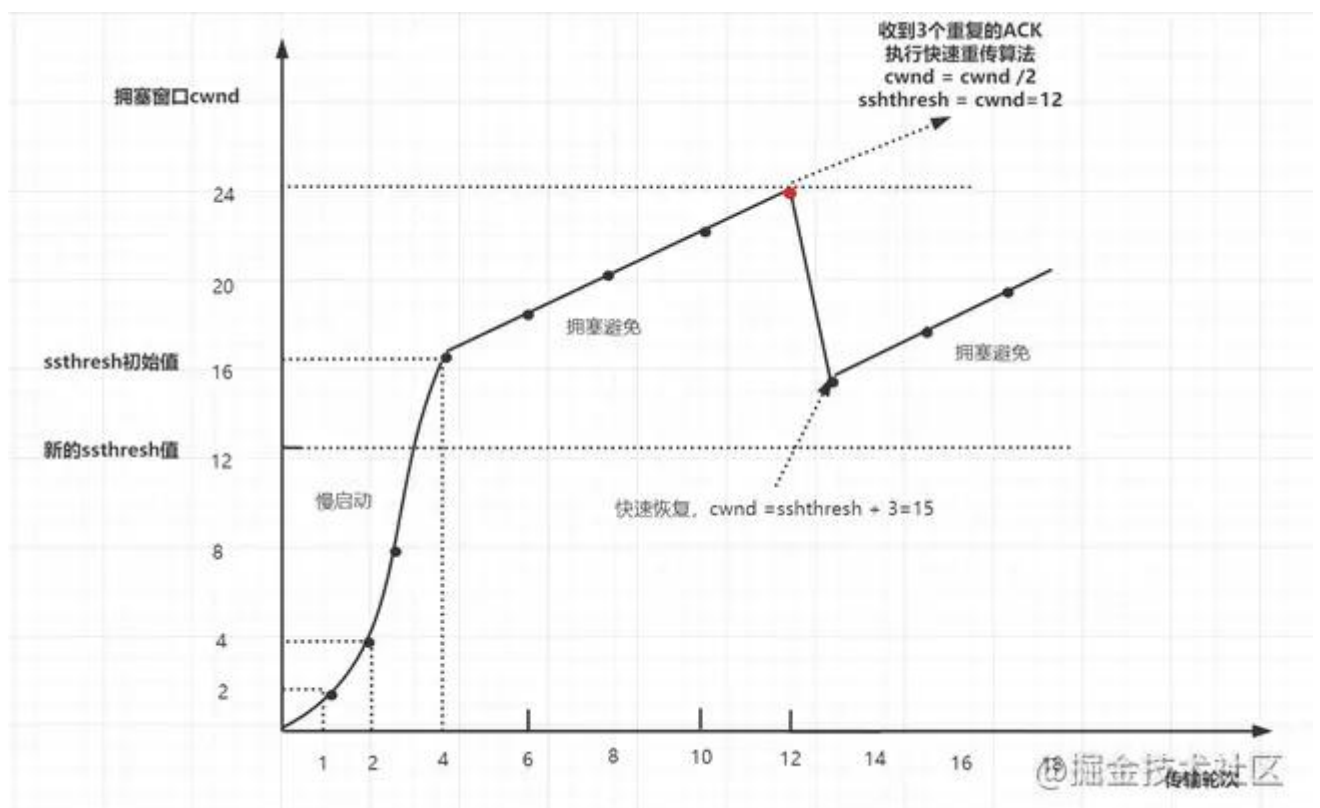
快速重传和快速恢复算法一般同时使用。快速恢复算法认为，还有 3 个重复 ACK 收到，说明网络也没那么糟糕，所以没有必要像 RTO 超时那么强烈。

正如前面所说，进入快速恢复之前， $cwnd$  和  $ssthresh$  已被更新：

- $cwnd = cwnd / 2$
- $ssthresh = cwnd$  复制代码

然后，真正的快速算法如下：

- $cwnd = ssthresh + 3$
- 重传重复的那几个 ACK（即丢失的那几个数据包）
- 如果再收到重复的 ACK，那么  $cwnd = cwnd + 1$
- 如果收到新数据的 ACK 后,  $cwnd = ssthresh$ 。因为收到新数据的 ACK，表明恢复过程已经结束，可以再次进入了拥塞避免的算法了。



### 31. TCP 和 UDP 分别对应的常见应用层协议有哪些？

基于 TCP 的应用层协议有：**HTTP、FTP、SMTP、TELNET、SSH**

- **HTTP**: HyperText Transfer Protocol (超文本传输协议)，默认端口 80
- **FTP**: File Transfer Protocol (文件传输协议)，默认端口(20 用于传输数据，21 用于传输控制信息)
- **SMTP**: Simple Mail Transfer Protocol (简单邮件传输协议)，默认端口 25
- **TELNET**: Teletype over the Network (网络电传)，默认端口 23
- **SSH**: Secure Shell (安全外壳协议)，默认端口 22

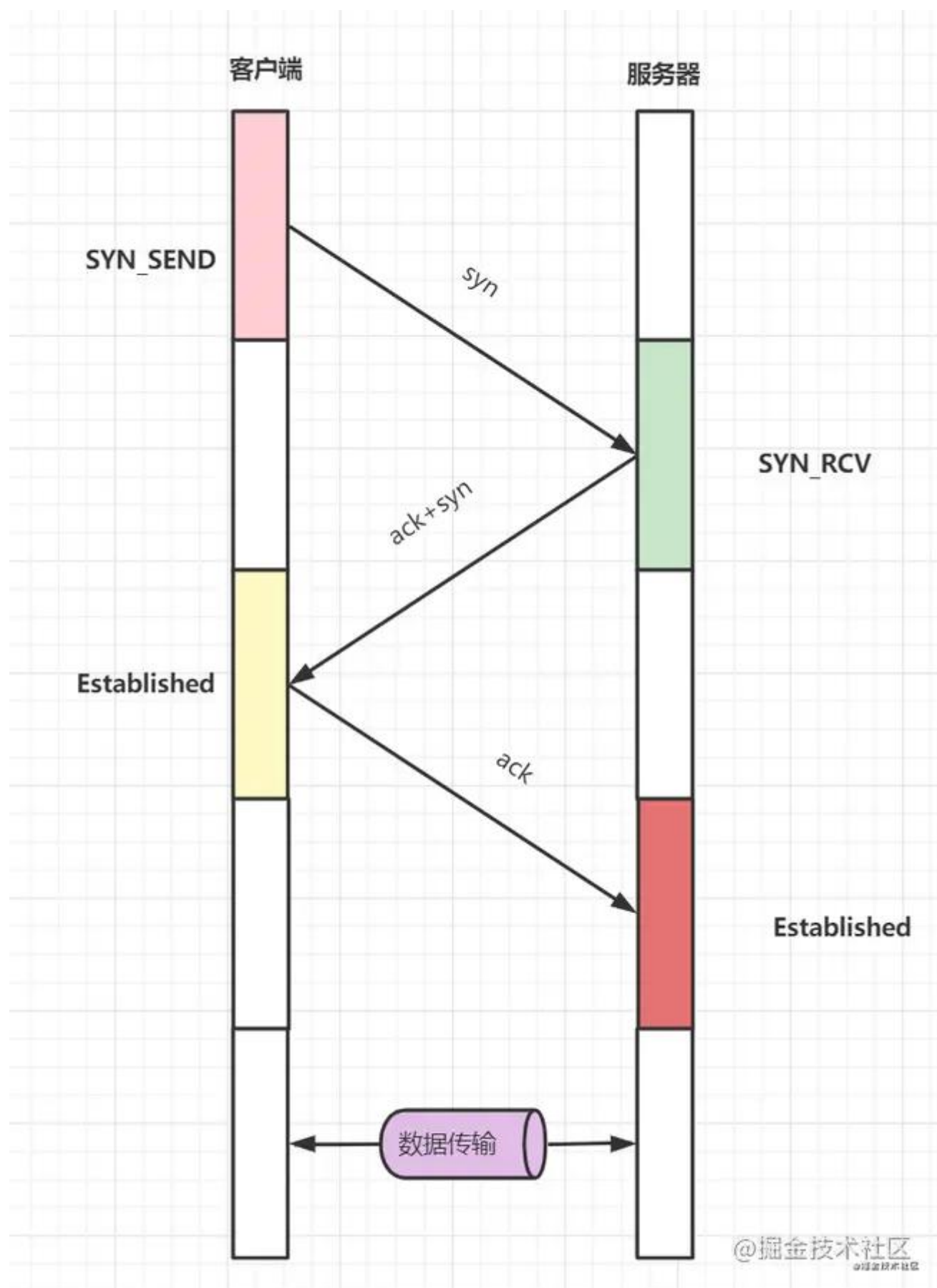
基于 UDP 的应用层协议：**DNS、TFTP、SNMP**

- **DNS**: Domain Name Service (域名服务)，默认端口 53
- **TFTP**: Trivial File Transfer Protocol (简单文件传输协议)，默认端口 69
- **SNMP**: Simple Network Management Protocol (简单网络管理协议)，通过 UDP 端口 161 接收，只有 Trap 信息采用 UDP 端口 162。

### 32. 半连接队列和 SYN Flood 攻击的关系

TCP 进入三次握手前，服务端会从 **CLOSED** 状态变为 **LISTEN** 状态，同时在内部创建了两个队列：半连接队列（**SYN** 队列）和全连接队列（**ACCEPT** 队列）。

什么是半连接队列（**SYN** 队列）呢？什么是全连接队列（**ACCEPT** 队列）呢？回忆下 TCP 三次握手的图：



- TCP 三次握手时，客户端发送 SYN 到服务端，服务端收到之后，便回复 **ACK 和 SYN**，状态由 **LISTEN 变为 SYN\_RCVD**，此时这个连接就被推入了 **SYN 队列**，即半连接队列。
- 当客户端回复 **ACK**，服务端接收后，三次握手就完成了。这时连接会等待被具体的应用取走，在被取走之前，它被推入 **ACCEPT 队列**，即全连接队列。

SYN Flood 是一种典型的 DoS (Denial of Service, 拒绝服务) 攻击，它在短时间内，伪造**不存在的 IP 地址**，向服务器大量发起 SYN 报文。当服务器回复 SYN+ACK 报文后，不会收到 ACK 回应报文，导致服务器上建立大量的半连接半连接队列满了，这就无法处理正常的 TCP 请求啦。

主要有 **syn cookie** 和 **SYN Proxy 防火墙**等方案应对。

•

**syn cookie:** 在收到 SYN 包后，服务器根据一定的方法，以数据包的源地址、端口等信息为参数计算出一个 **cookie** 值作为自己的 SYNACK 包的序列号，回复 SYN+ACK 后，服务器并不立即分配资源进行处理，等收到发送方的 ACK 包后，重新根据数据包的源地址、端口计算该包中的确认序列号是否正确，如果正确则建立连接，否则丢弃该包。

•

•

**SYN Proxy 防火墙:** 服务器防火墙会对收到的每一个 SYN 报文进行代理和回应，并保持半连接。等发送方将 ACK 包返回后，再重新构造 SYN 包发到服务器，建立真正的 TCP 连接。

•

### 33. 有了 IP 地址，为什么还要用 MAC 地址？

- 简而言之，标识网络中的一台计算机，比较常用的就是 **IP 地址和 MAC 地址**，但计算机的 IP 地址可由用户自行更改，管理起来就相对困难，而 MAC 地址不可更改，所以一般会把 IP 地址和 MAC 地址组合起来使用。
- 那只使用 MAC 地址不用 IP 地址行不行呢？不行的！因为最早就是 MAC 地址先出现的，并且当时并不用 IP 地址，只用 MAC 地址，后来随着网络中的设备越来越多，整个路由过程越来越复杂，便出现了子网的概念。对于目的地址在其他子网的数据包，路由只需要将数据包送到那个子网即可。
- 那为什么要用 IP 地址呢？是因为 IP 地址是和地域相关的，对于同一个子网上的设备，IP 地址的前缀都是一样的，这样路由器通过 IP 地址的前缀就知道设备在在哪个子网上了，而只用 MAC 地址的话，路由器则需要记住每个 MAC 地址

在哪个子网，这需要路由器有极大的存储空间，是无法实现的。

- IP 地址可以比作为地址，MAC 地址为收件人，在一次通信过程中，两者是缺一不可的。

### 34. 聊聊保活计时器的作用

除时间等待计时器外，TCP 还有一个保活计时器（keepalive timer）。设想这样的场景：客户已主动与服务器建立了 TCP 连接。但后来客户端的主机突然发生故障。显然，服务器以后就不能再收到客户端发来的数据。因此，应当有措施使服务器不要再白白等待下去。这就需要使用保活计时器了。

服务器每收到一次客户的数据，就重新设置保活计时器，时间的设置通常是两个小时。若两个小时都没有收到客户端的数据，服务端就发送一个探测报文段，以后则每隔 75 秒钟发送一次。若连续发送 10 个探测报文段后仍然无客户端的响应，服务端就认为客户端出了故障，接着就关闭这个连接。

### 35. 聊聊 ARP 协议

ARP 协议，地址解析协议，是一个由 IP 地址获取 MAC 物理地址的 TCP/IP 协议。

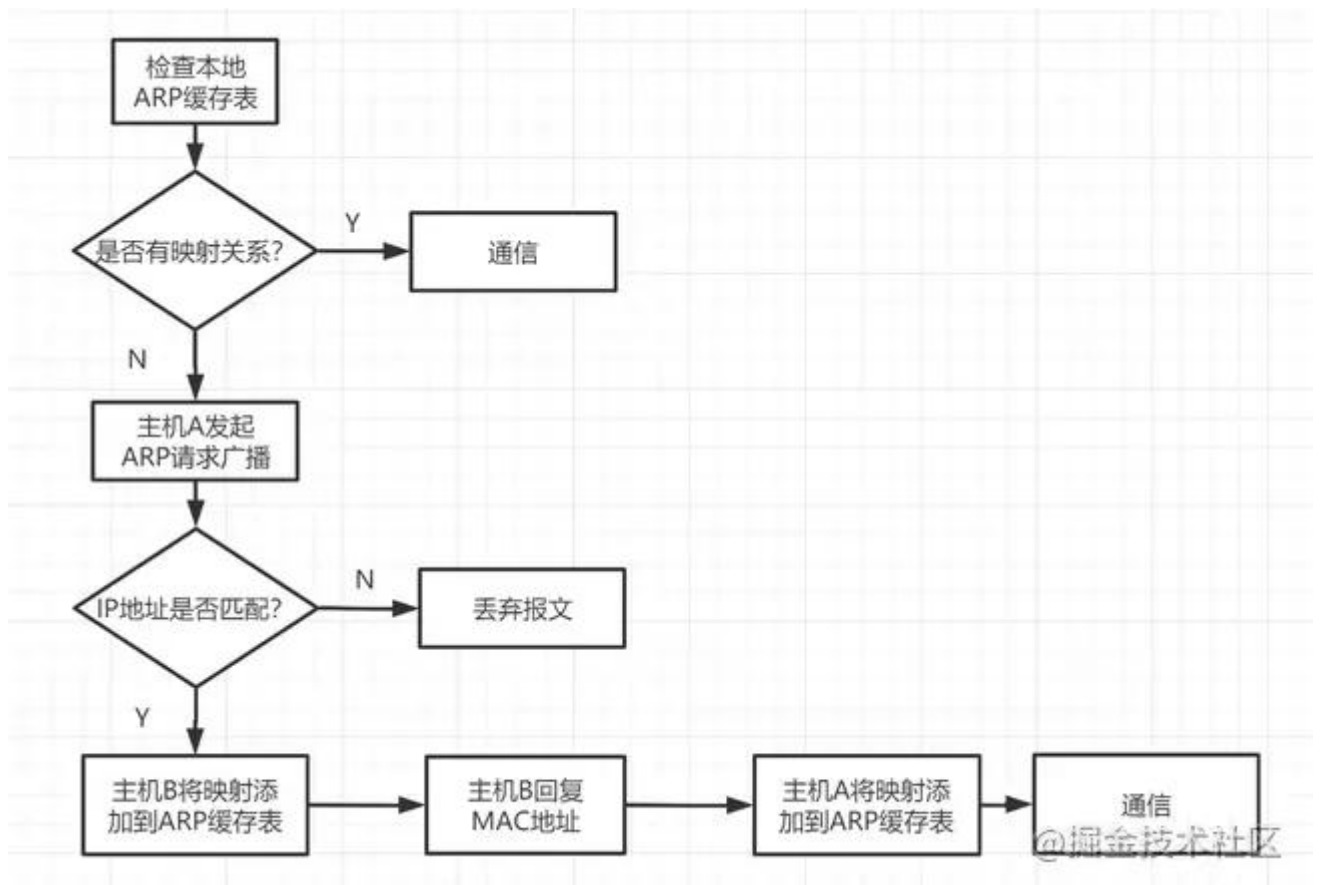
**什么是 IP 地址，什么是 MAC 地址？**

- IP 地址：是互联网协议地址，它是 IP 协议提供的一种统一的地址格式，它为互联网上的每一个网络和每一台主机分配一个逻辑地址，以此来屏蔽物理地址的差异。
- MAC 地址：以太网地址或物理地址，它是一个用来确认网络设备位置的位址。

**为什么需要 ARP 协议呢？**

- 在网络访问层中，同一局域网中的一台主机要和另一台主机进行通信，需要通过 MAC 地址进行定位，然后才能进行数据包的发送。
- 而在网络层和传输层中，计算机之间是通过 IP 地址定位目标主机，对应的数据报文只包含目标主机的 IP 地址，而没有 MAC 地址。
- 因此，在发送之前需要根据 IP 地址获取 MAC 地址，然后才能将数据包发送到正确的目标主机，而这个获取过程是通过 ARP 协议完成的。

**ARP 的工作流程**

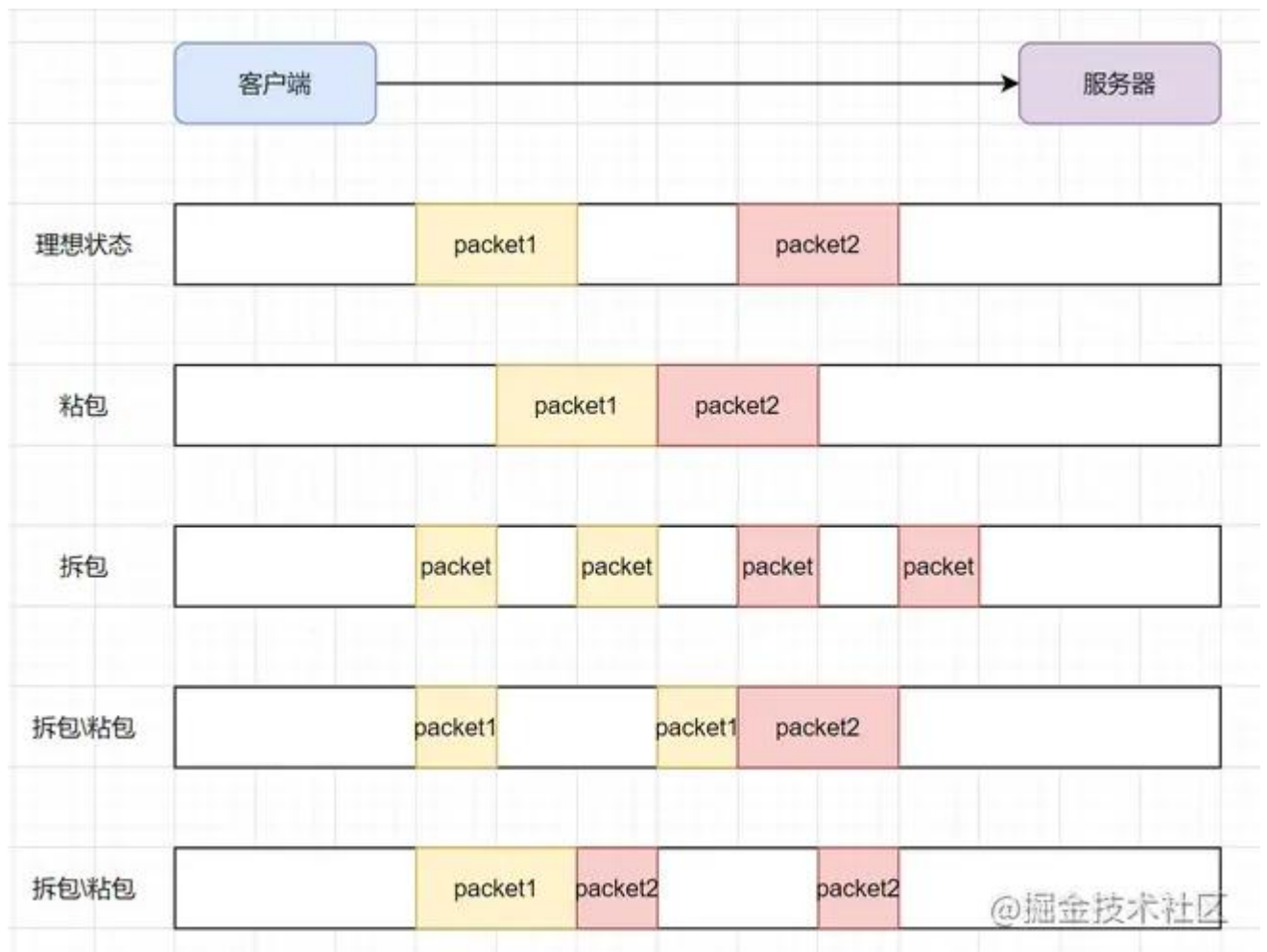


当主机 A 与主机 B 要通信时，工作流程如下：

1. 查询本地 ARP 缓存表,看是否有 IP 地址及其对应的 MAC 地址。
2. 如果没匹配到主机 B 的 MAC 地址，主机 A 会在局域网内广播发送一个 ARP 请求分组，局域网内所有主机都会收到该请求分组。
3. 主机 B 收到请求分组报文，发现报文中的 IP 与自己匹配，就 A 的 IP 和 MAC 地址添加到本地 ARP 缓存表中。
4. 主机 B 向主机 A 响应一个含自身 MAC 地址的报文。
5. 主机 A 收到报文后，将 B 的 IP 和 MAC 地址添加至 ARP 缓存表中。

### 36. TCP 的粘包和拆包

TCP 是面向流，没有界限的一串数据。TCP 底层并不了解上层业务数据的具体含义，它会根据 TCP 缓冲区的实际情况进行包的划分，所以在业务上认为，一个完整的包可能会被 TCP 拆分成多个包进行发送，也有可能把多个小的包封装成一个大的数据包发送，这就是所谓的 TCP 粘包和拆包问题。



### 为什么会产生粘包和拆包呢?

- 要发送的数据小于 TCP 发送缓冲区的大小，TCP 将多次写入缓冲区的数据一次发送出去，将会发生粘包；
- 接收数据端的应用层没有及时读取接收缓冲区中的数据，将发生粘包；
- 要发送的数据大于 TCP 发送缓冲区剩余空间大小，将会发生拆包；
- 待发送数据大于 MSS（最大报文长度），TCP 在传输前将进行拆包。即 TCP 报文长度-TCP 头部长度>MSS。

### 解决方案:

- 发送端将每个数据包封装为固定长度
- 在数据尾部增加特殊字符进行分割
- 将数据分为两部分，一部分是头部，一部分是内容体；其中头部结构大小固定，且有一个字段声明内容体的大小。

### 37. forward 和 redirect 的区别?

- **直接转发方式（Forward）**，客户端和浏览器只发出一次请求，Servlet、HTML、JSP 或其它信息资源，由第二个信息资源响应该请求，在请求对象 request 中，保存的对象对于每个信息资源是共享的。
- **间接转发方式（Redirect）** 实际是两次 HTTP 请求，服务器端在响应第一次请求的时候，让浏览器再向另外一个 URL 发出请求，从而达到转发的目的。

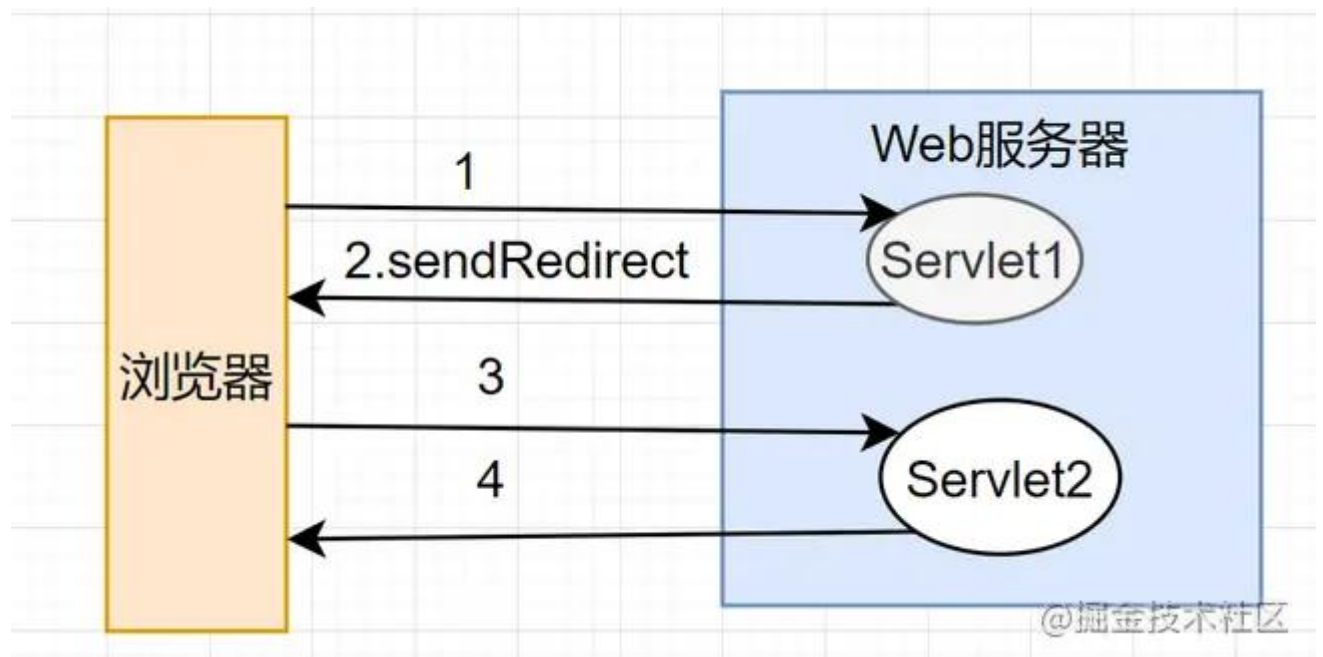
举个通俗的例子：

- 直接转发就相当于：“A 找 B 借钱，B 说没有，B 去找 C 借，借到借不到都会把消息传递给 A”；
- 间接转发就相当于：“A 找 B 借钱，B 说没有，让 A 去找 C 借”。

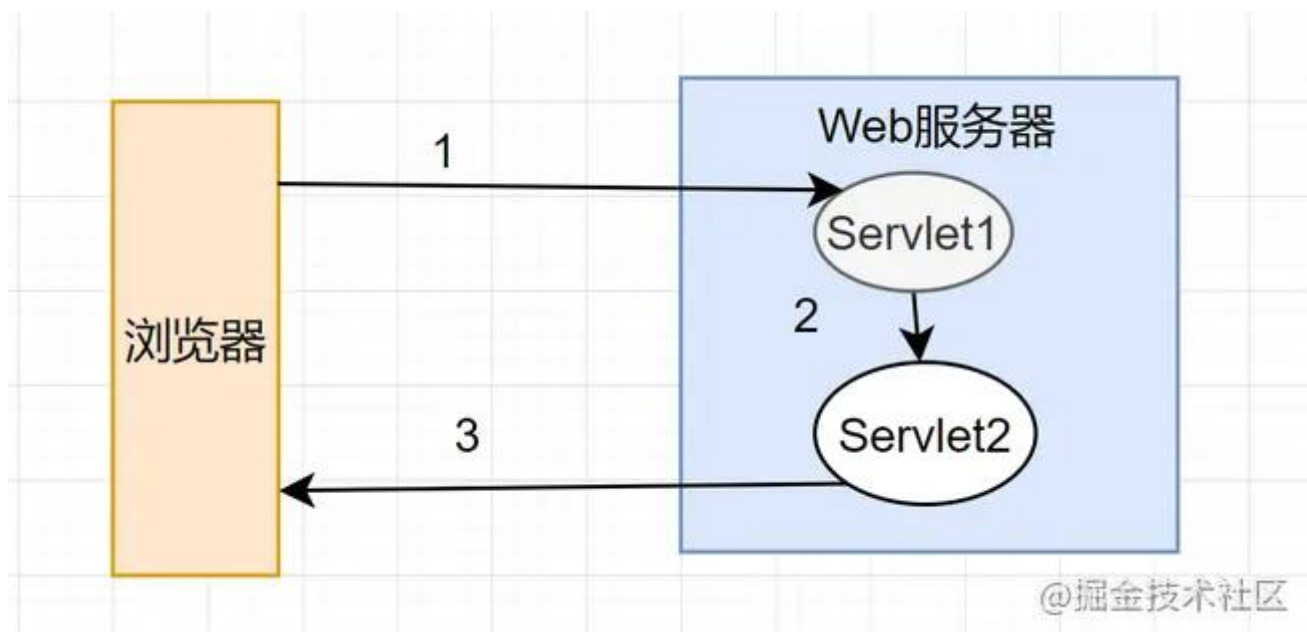
\*\*

看这两个图，可以更容易理解一些：

- Redirect 的工作原理：



- forward 的工作原理



### 38. Nagle 算法与延迟确认

#### Nagle 算法

如果发送端疯狂地向接收端发送很小的包，比如就 1 个字节，那么亲爱的小伙伴，你们觉得会有什么问题呢？

TCP/IP 协议中，无论发送多少数据，总是要在数据前面加上协议头，同时，对方接收到数据，也需要发送 ACK 表示确认。为了尽可能的利用网络带宽，TCP 总是希望尽可能的发送足够大的数据。**Nagle 算法**就是为了尽可能发送大块数据，避免网络中充斥着许多小数据块。

**Nagle 算法的基本定义是：任意时刻，最多只能有一个未被确认的小段。**所谓“小段”，指的是小于 MSS 尺寸的数据块，所谓“未被确认”，是指一个数据块发送出去后，没有收到对方发送的 ACK 确认该数据已收到。

**Nagle 算法的实现规则：**

- 如果包长度达到 MSS，则允许发送；
- 如果该包含有 FIN，则允许发送；
- 设置了 TCP\_NODELAY 选项，则允许发送；
- 未设置 TCP\_CORK 选项时，若所有发出去的小数据包（包长度小于 MSS）均被确认，则允许发送；
- 上述条件都未满足，但发生了超时（一般为 200ms），则立即发送。

#### 延迟确认

如果接受方刚接收到发送方的数据包，在很短很短的时间内，又接收到第二个包。那么请问接收方是一个一个地回复好点，还是合并一起回复好呢？

接收方收到数据包后，如果暂时没有数据要发给对端，它可以等一段时再确认（Linux 上默认是 40ms）。如果这段时间刚好有数据要传给对端，ACK 就随着数据传输，而不需要单独发送一次 ACK。如果超过时间还没有数据要发送，也发送 ACK，避免对端以为丢包。

但是有些场景不能延迟确认，比如发现了乱序包、接收到了大于一个 **frame** 的报文，且需要调整窗口大小等。

一般情况下，**Nagle 算法**和**延迟确认**不能一起使用，**Nagle 算法**意味着延迟发，**延迟确认**意味着延迟接收，酱紫就会造成更大的延迟，会产生性能问题。

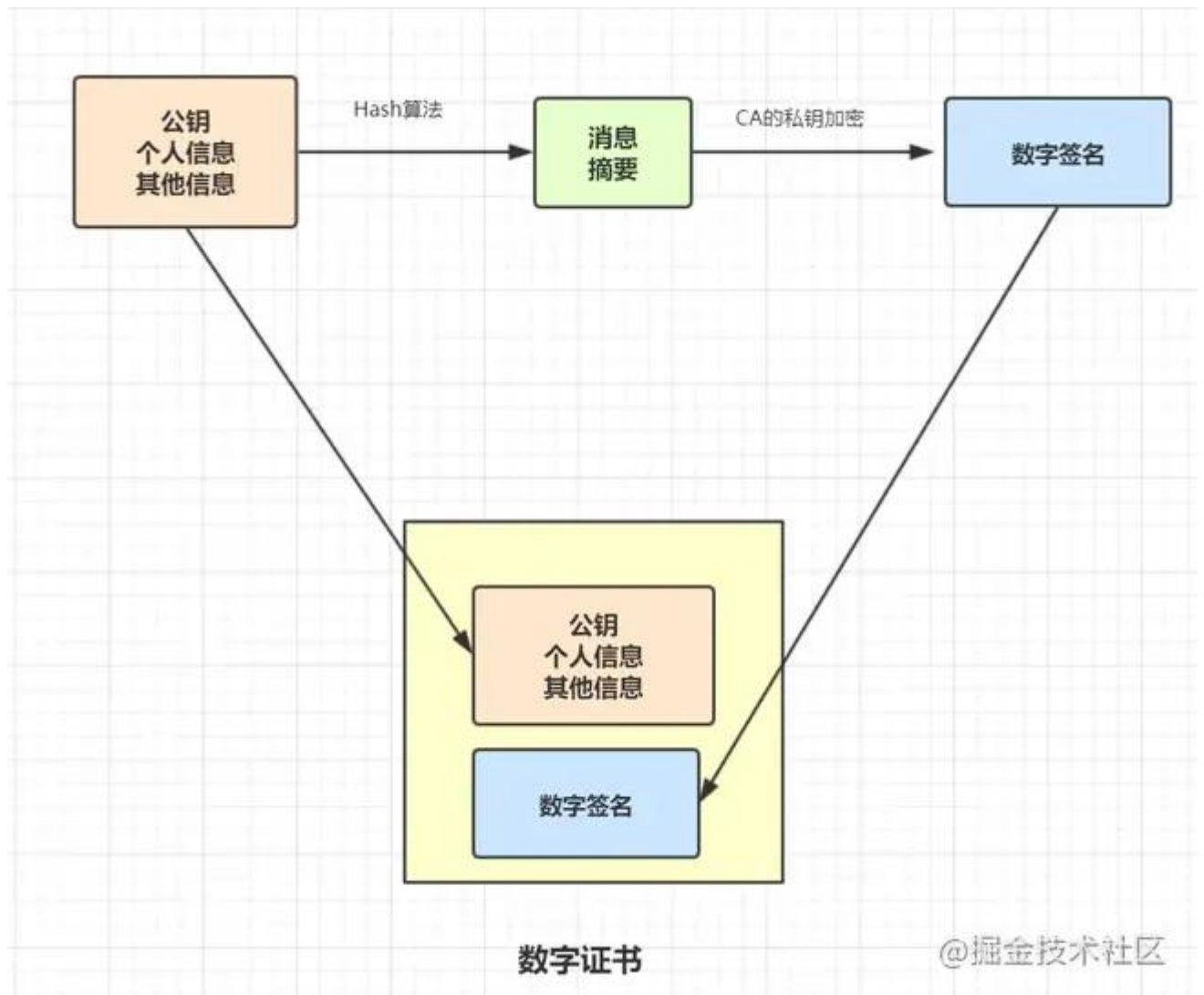
### 39. URI 和 URL 的区别

- URI，全称是 Uniform Resource Identifier)，中文翻译是统一资源标志符，主要作用是唯一标识一个资源。
- URL，全称是 Uniform Resource Location)，中文翻译是统一资源定位符，主要作用是提供资源的路径。

打个经典比喻吧，URI 像是身份证，可以唯一标识一个人，而 URL 更像一个住址，可以通过 URL 找到这个人。

### 40. 什么是数字签名？ 什么是数字证书？

了解过 **Https 原理**的小伙伴，都知道数字证书这玩意。为了避免公钥被篡改，引入了数字证书，如下：



## 数字证书构成

- 公钥和个人信息，经过 Hash 算法加密，形成消息摘要；将消息摘要拿到拥有公信力的认证中心（CA），用它的私钥对消息摘要加密，形成数字签名。
- 公钥和个人信息、数字签名共同构成数字证书。

## 41. 什么是 SQL 注入？举个例子？

SQL 注入是一种代码注入技术，一般被应用于攻击 web 应用程序。它通过在 web 应用接口传入一些特殊参数字符，来欺骗应用服务器，执行恶意的 SQL 命令，以达到非法获取系统信息的目的。它目前是黑客对数据库进行攻击的最常用手段之一。

## SQL 注入是如何攻击的？

举个常见的**业务场景**：在 web 表单搜索框输入员工名字，然后后台查询出对应名字的员工。

员工查询

田螺

搜索

捡田螺的小男孩

@掘金技术社区

这种场景下，一般都是前端页面把一个名字参数 `name` 传到后台，然后后台通过 SQL 把结果查询出来

```
name = "田螺"; //前端传过来的
```

```
SQL= "select * from staff where name=" + name; //根据前端传过来的 name 参数，查询数据库员工表 staff 复制代码
```

因为 SQL 是直接拼接的，如果我们完全信任前端传的参数说的话。假如前端传这么一个参数时 `' or '1'='1'`，SQL 就变成酱紫的啦。

```
select * from staff where name=' or '1'='1';复制代码
```

这个 SQL 会把所有的员工信息全都查出来了，酱紫就请求用户已经越权啦。请求者可以获取所有员工的信息，信息已经暴露了啦。

如何预防 SQL 注入问题

## 1). 使用#{ }而不是\${ }

在 MyBatis 中,使用#{ }而不是\${ }, 可以很大程度防止 sql 注入。

- 因为#{ }是一个参数占位符,对于字符串类型,会自动加上"",其他类型不加。由于 Mybatis 采用**预编译**,其后的参数不会再进行 SQL 编译,所以一定程度上防止 SQL 注入。
- \${ }是一个简单的字符串替换,字符串是什么,就会解析成什么,存在 SQL 注入风险

## 2). 不要暴露一些不必要的日志或者安全信息,比如避免直接响应一些 sql 异常信息。

如果 SQL 发生异常了,不要把这些信息暴露响应给用户,可以自定义异常进行响应

## 3). 不相信任何外部输入参数,过滤参数中含有的一些数据库关键词关键词

可以加个参数校验过滤的方法,过滤 union, or 等数据库关键词

## 4). 适当的权限控制

在你查询信息时,先校验下当前用户是否有这个权限。比如说,实现代码的时候,可以让用户多传一个企业 id 什么的,或者获取当前用户的 session 信息等,在查询前,先校验一下当前用户是否是这个企业下的等等,是的话才有这个查询员工的权限。

## 42. 什么是 DoS、DDoS、DRDoS 攻击?

- **DOS:** (Denial of Service),中文名称是拒绝服务,一切能引起 DOS 行为的攻击都被称为 DOS 攻击。最常见的 DoS 攻击有计算机网络宽带攻击和连通性攻击。
- **DDoS:** (Distributed Denial of Service),中文名称是分布式拒绝服务。是指处于不同位置的多个攻击者同时向一个或数个目标发动攻击,或者一个攻击者控制了位于不同位置的多台机器并利用这些机器对受害者同时实施攻击。常见的 DDos 有 SYN Flood、Ping of Death、ACK Flood、UDP Flood 等。
- **DRDoS:** (Distributed Reflection Denial of Service), 中文名称是分布式反射拒绝服务,该方式靠的是发送大量带有被害者 IP 地址的数据包给攻击主机,然后攻击主机对 IP 地址源做出大量回应,形成拒绝服务攻击。

## 43. WebSocket 与 socket 的区别

- Socket = IP 地址 + 端口 + 协议。

具体来说，Socket 是一套标准，它完成了对 TCP/IP 的高度封装，屏蔽网络细节以方便开发者更好地进行网络编程。

- WebSocket 是一个持久化的协议，它是伴随 HTTP5 而出的协议，用来解决 **http 不支持持久化连接** 的问题。
- Socket 一个是**网编编程的标准接口**，而 WebSocket 是应用层通信协议。

## 44. ICMP 协议的功能

ICMP,Internet Control Message Protocol ,Internet 控制消息协议。

- ICMP 协议是一种面向无连接的协议，用于传输出错报告控制信息。
- 它是一个非常重要的协议，它对于网络安全具有极其重要的意义。它属于网络层协议，主要用于在主机与路由器之间传递控制信息，包括**报告错误、交换受限控制和状态信息**等。
- 当遇到 IP 数据无法访问目标、IP 路由器无法按当前的传输速率转发数据包等情况时，会自动发送 ICMP 消息。

比如我们日常使用得比较多的 **ping**，就是基于 ICMP 的。

## 45. Http 请求的过程与原理

HTTP 是一个基于 TCP/IP 协议来传递数据的超文本传输协议，传输的数据类型有 HTML 文件、图片文件等。以访问百度有例子，看下一次 Http 的请求过程



1. 客户端进行 DNS 域名解析，得到对应的 IP 地址
2. 根据这个 IP，找到对应的服务器建立连接（三次握手）

3. 建立 TCP 连接后发起 HTTP 请求（一个完整的 http 请求报文）
4. 服务器响应 HTTP 请求，客户端得到 html 代码
5. 客户端解析 html 代码，用 html 代码中的资源(如 js,css,图片等等)渲染页面。
6. 服务器关闭 TCP 连接（四次挥手）

## 46. 说下 ping 的原理

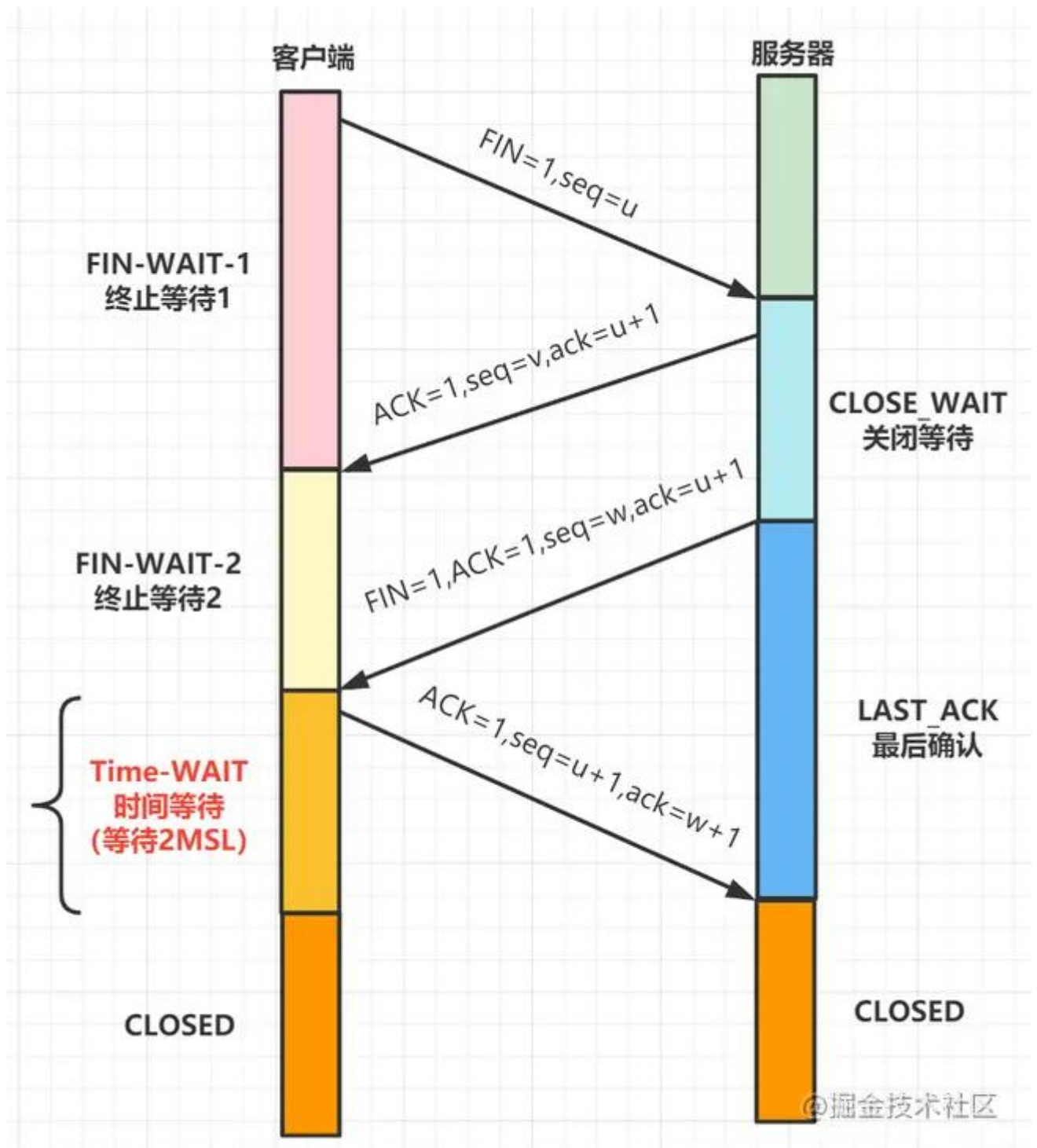
ping, **Packet Internet Groper**, 是一种因特网包探索器, 用于测试网络连接量的程序。Ping 是工作在 TCP/IP 网络体系结构中应用层的一个服务命令, 主要是向特定的目的主机发送 ICMP (Internet Control Message Protocol 因特网报文控制协议) 请求报文, 测试目的站是否可达及了解其有关状态

一般来说, ping 可以用来检测网络通不通。它是基于 ICMP 协议工作的。假设机器 A ping 机器 B, 工作过程如下:

1. ping 通知系统, 新建一个固定格式的 ICMP 请求数据包
2. ICMP 协议, 将该数据包和目标机器 B 的 IP 地址打包, 一起转交给 IP 协议层
3. IP 层协议将本机 IP 地址为源地址, 机器 B 的 IP 地址为目标地址, 加上一些其他的控制信息, 构建一个 IP 数据包
4. 先获取目标机器 B 的 MAC 地址。
5. 数据链路层构建一个数据帧, 目的地址是 IP 层传过来的 **MAC 地址**, 源地址是本机的 **MAC 地址**
6. 机器 B 收到后, 对比目标地址, 和自己本机的 MAC 地址是否一致, 符合就处理返回, 不符合就丢弃。
7. 根据目的主机返回的 ICMP 回送回答报文中的时间戳, 从而计算出往返时间
8. 最终显示结果有这几项: 发送到目的主机的 IP 地址、发送 & 收到 & 丢失的分组数、往返时间的最小、最大& 平均值

## 47. 如果服务器出现了大量 CLOSE\_WAIT 状态如何解决。

我们先来回忆下 TCP 的四次挥手



- 服务器端收到客户端发送的 `FIN` 后，TCP 协议栈就会自动发送 `ACK`，接着进入 `CLOSE_WAIT` 状态。
- 但是如果服务器端不执行 `socket` 的 `close()` 操作，那么就没法进入 `LAST_ACK`，导致大量连接处于 `CLOSE_WAIT` 状态
- 所以，如果服务器出现了大量 `CLOSE_WAIT` 状态，一般是程序 Bug，或者关闭 `socket` 不及时。

## 48. 什么是 CSRF 攻击，如何避免

### 什么是 CSRF 攻击？

CSRF，跨站请求伪造（英语：Cross-site request forgery），简单点说就是，攻击者盗用了你的身份，以你的名义发送恶意请求。跟跨网站脚本（XSS）相比，XSS 利用的是用户对指定网站的信任，CSRF 利用的是网站对用户网页浏览器的信任。

### CSRF 是如何攻击的呢？

我们来看下这个例子哈（来自百度百科）

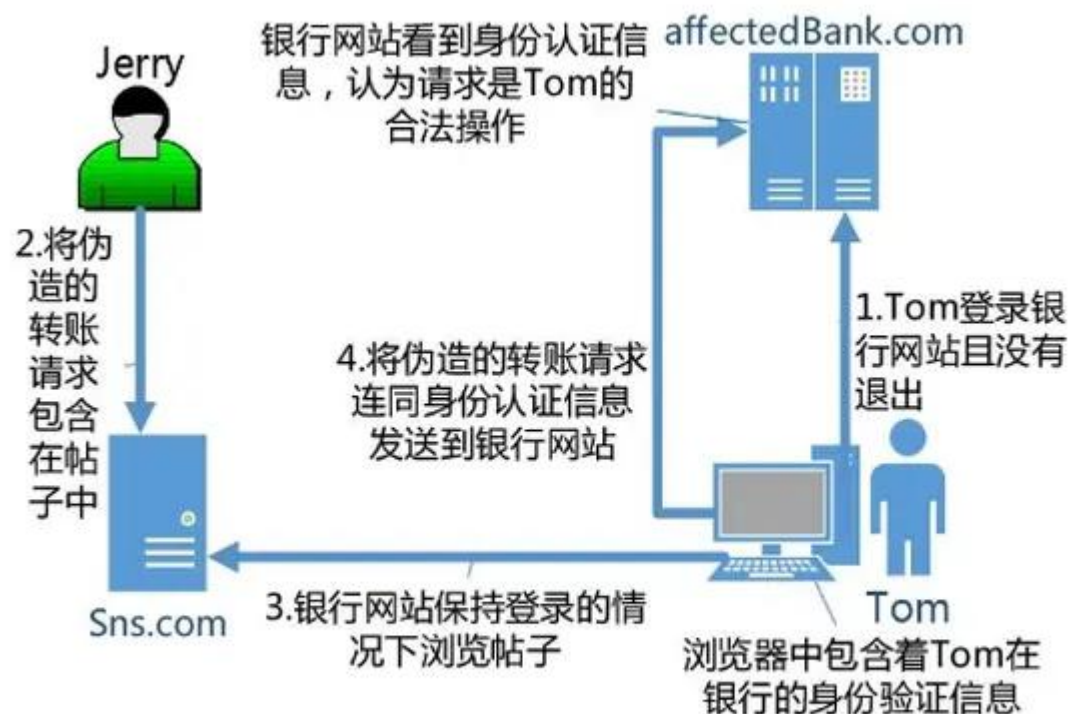


图 1 典型 CSRF 的例子

@掘金技术社区

- 1. Tom 登陆银行，没有退出，浏览器包含了 Tom 在银行的身份认证信息。
- 1. 黑客 Jerry 将伪造的转账请求，包含在在帖子
-

1. Tom 在银行网站保持登陆的情况下，浏览帖子

•

1. 将伪造的转账请求连同身份认证信息，发送到银行网站

•

1. 银行网站看到身份认证信息，以为就是 Tom 的合法操作，最后造成 Tom 资金损失。

### 如何解决 CSRF 攻击

- 检查 Referer 字段。HTTP 头中有一个 Referer 字段，这个字段用以标明请求来源于哪个地址。
- 添加校验 token。

### 49. RARP 协议的工作原理？

- ARP(地址解析协议),是设备通过自己知道的 IP 地址来获得自己不知道的物理地址的协议。
- RARP(反向地址转换协议)以与 ARP 相反的方式工作。RARP 发出要反向解析的物理地址并希望返回其对应的 IP 地址，应答包括由能够提供所需信息的 RARP 服务器发出的 IP 地址。（应用于无盘机）

RARP 工作原理如下：

1. 发送主机发送一个本地的 RARP 广播，在此广播包中，声明自己的 MAC 地址并且请求任何收到此请求的 RARP 服务器分配一个 IP 地址；
2. 本地网段上的 RARP 服务器收到此请求后，检查其 RARP 列表，查找该 MAC 地址对应的 IP 地址；
3. 如果存在，RARP 服务器就给源主机发送一个响应数据包并将此 IP 地址提供给对方主机使用；
4. 如果不存在，RARP 服务器对此不做任何的响应；
5. 源主机收到从 RARP 服务器的响应信息，就利用得到的 IP 地址进行通讯；如果一直没有收到 RARP 服务器的响应信息，表示初始化失败。

### 50. 了解下 DNS，解析过程？

DNS, **domain name system**, 域名解析系统，是因特网上作为域名和 IP 地址相互映射的一个分布式数据库。它的作用非常简单，就是可以根据域名查出对应的 IP 地址。

解析过程如下：

1. 首先，检查**浏览器缓存**中，查找对应的 IP 地址，找到就直接返回；否则下一步。
2. 将请求发送给**本地 DNS 服务器**，在本地 DNS 服务器缓存中查询，如果查找到就直接返回，否则下一步；
3. 本地 DNS 服务器向**根域名服务器**发送请求，根域名服务器会告诉本地 DNS 服务器去查询**哪个顶级域名服务器**。
4. 本地域名服务器向顶级域名服务器发起查询请求，顶级域名服务器会告诉本地 DNS 服务器，去查找**哪个权限域名服务器**。
5. 本地域名服务器向权限域名服务器发起查询请求，权限域名服务器告诉本地域名服务器请求域名所对应的 IP 地址。
6. 最后，本地域名服务器告诉主机**请求域名**所对应的 IP 地址。

比如要查询 [www.baidu.com](http://www.baidu.com) 的 IP 地址：

- 首先会在浏览器的缓存中，是否查找到 [www.baidu.com](http://www.baidu.com) 的对应的 IP，找到就直接返回；否则下一步。
- 将请求发送给本地 DNS 服务器，在本地 DNS 服务器缓存中查询，如果查找到就直接返回，否则下一步；
- 本地 DNS 服务器向**根域名服务器**发送请求，根域名服务器返回负责 [www.baidu.com](http://www.baidu.com) 的顶级域名服务器的 IP 地址的列表。
- 本地 DNS 服务器再向其中一个负责 [www.baidu.com](http://www.baidu.com) 的顶级域名服务器发送一个请求，返回负责 [www.baidu.com](http://www.baidu.com) 的权威域名服务器的 IP 地址列表。
- 本地 DNS 服务器再向其中一个权威域名服务器发送一个请求，返回 [www.baidu.com](http://www.baidu.com) 所对应的 IP 地址。

作者：捡田螺的小男孩

链接：<https://juejin.cn/post/6988794419910541348>